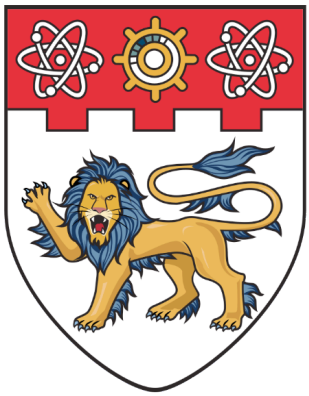


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

Augmenting Rather Than Replacing Artists with Generative AI

Zhu Zhanyan
U2320498K
AY 2026/2027

FYP ID: CCDS25-0989
Supervisor: Prof. Chee Wei Tan
Examiner:

Abstract

The role of the artist is questioned in the age of AI Image Generation. This project demonstrates a model of AI-Artist co-creation that allows artist and AI to coexist through the development of an iPad AI-Assisted Drawing App, Broche. Broche differs from existing AI Assisted solutions by the introduction of a conversational Sketch Agent for interacting with its AI features and secure handling of Online Inference API credentials. Broche also introduces iterative refinement of AI generated images and a familiar drawing UI to align with existing artist workflows. The project demonstrates that Online Model Inference is still necessary for resource constrained devices, despite recent interest in on-device AI. The App capabilities and limitations are demonstrated on Illustration, Graphic Design and UI Design. Broche paves the way for future work to enhance existing artist workflows with AI models. Source code for the project is available at https://github.com/mrzzy/ntu-fyp.git.

Acknowledgements

I express my gratitude to Professor Chee Wei Tan for supervising me in the development of this project. His guidance and insights were invaluable in ensuring the timely completion of this project.

Special Thanks is given to the School of Science and Technology, for extending Apple Developer Program access to Alumni Students.

My sincere appreciation is given to friends and family who have given me support and encouragement throughout the process.

Table of Contents

1. Introduction **4**
1.1. Background 4
1.1.1. Advances in AI Image Generation 4
1.1.2. Role of the Artist in the Age of AI 4
1.1.3. AI-Artist Co-Creation 4
1.2. Problem & Scope 5
1.2.1. Limitations with Existing Solutions 5
1.2.2. Objectives 5
1.2.3. Key Contributions 6
2. Literature Review **7**
2.1. Advances in AI 7
2.1.1. Text Generation Models 7
2.1.2. Visual Language Models 7
2.1.3. Diffusion Models 7
2.1.4. AI Model Selection 8
2.1.5. AI Agents 8
2.2. Software Engineering 9
2.2.1. Software Engineering Practices 9
2.2.2. iOS Development 9
2.2.3. Firebase 9
2.2.4. AI Model Deployment 9
2.3. Product Design 11
2.3.1. End User Workflow 11
2.3.2. Competitor Analysis 11
3. Analysis and Design Approach **12**
3.1. Product Design 12
3.2. Software Architecture 13
3.3. AI Model Selection 13
3.4. AI Model Deployment 14
3.5. Sketch Agent 18
4. Detailed Implementation **19**
4.1. UI Prototype 19
4.2. Data Modeling & Persistence 22
4.3. Performance Optimisation 23
4.4. AI Model Selection 24
4.5. AI Model Deployment 25
4.6. Sketch Agent 25
4.6.1. Agent Workflows 26
4.6.2. Agent Tools 26
5. Results **30**
6. Discussion **32**
6.1. Implications 32

6.2. Limitations 32
6.3. Future Work 33
7. Conclusion 34
8. References 34

1. Introduction

1.1. Background

1.1.1. Advances in AI Image Generation

Image Generation Artificial Intelligence (AI) models are now capable of generating high fidelity images from visual and text prompts. As a testament to advances in image generation quality, Van Hees et al. demonstrated that humans largely prefer AI generated content over their human counterparts [1]. AI is also sufficiently capable of generating artworks. Famously, a Midjourney Image Generation model was used to craft "Théâtre D'opéra Spatial" that clinched an regional art award in the US [2]. In 2018, "Edmond de Belamy", a classical portrait generated by an AI model was auctioned off at an eye watering $432,500 [3].

Figure 1. "Théâtre D'opéra Spatial" by Jason M. Allen with assistance from Midjourney
Figure 2. "Shrimp Jesus," an example of "AI slop", generated by Bing Image

1.1.2. Role of the Artist in the Age of AI

The role of the artist in the age of AI is now in question since AI models can generate hyperrealistic images of various styles. Should AI perform most of the creation process or is human invention still necessary in the creative process?

Purely human generated works do not take sufficient advantage of the recent efficiency gains of AI image generation. A human artist working for weeks or months on a piece cannot come close to the pace of an Image Generation model that can craft a piece end-to-end in a few seconds [4]. AI must be integrated into existing artist's workflows in order to stay competitive in this rapidly evolving market.

Can we remove the human from the equation? Fully or mostly AI generated works are undesirable as they cannot receive copyright protections as is the case for Jason M. Allen's "Théâtre D'opéra Spatial" [5]. Pure AI creation also results in works such as "Shrimp Jesus" that are aesthetically pleasing but conceptually vacant, creating a deluge of AI "Slop", low quality content that is sweeping into our online spaces [6].

1.1.3. AI-Artist Co-Creation

Naturally, this points us in the direction of AI-Artist co-creation. Anscomb argues that creation of AI-assisted works can be seen as AI-assisted production where AI is used solely as a production tool to realise a human directed vision [7]. With a human involved in the development of a piece, significantly reduces the time taken for the artist to create while curtailing the tendency of AI to generate mindless content.

1.2. Problem & Scope

1.2.1. Limitations with Existing Solutions

Several existing attempts [8], [9], [10], [11], [12] have been made to develop an effective User Interface (UI) for visual graphical creation with AI image models. However, their capabilities are insufficient due to three major limitations.

Existing solutions rely on the human user's ability to craft a clear and unambiguous text prompt [13]to convey all the style and rendering attributes that the AI image model has to replicate while the human user has a tendency to be concise. For example a human user might type "warm sunset" when "Warm sunset mood with golden hour tones, soft orange and pink gradients, glowing highlights, ambient light casting gentle shadows, soft focus background, minimalistic style with warm color palette" is more adequate.

These solutions also offer limited support for artists to iteratively refine their work, which conflicts with the iterative workflows through which artists progressively develop a final piece [15]. Instead, most applications generate final images single shot from image and/or text prompts as an end to end rendering process. Once an image is generated, options for the artist to edit, alter or otherwise take the image in a different design direction is limited. Some regional editing is available via inpainting in the Wand App. However, the burden falls on the user to create an accurate inpainting mask and text conditioning prompt [12] Modern Image Generation Models such as Qwen-Image have demonstrated strong image editing capabilities without relying on clear user-specified masks [16] have yet to be applied in iterative artistic creation.

The bare bones UI provided by some of these solutions insufficiently signed with existing artistic workflows. Several solutions [8], [9], [12] provide UI controls that map directly to AI model inputs, providing text boxes for prompt input, slides for generation parameters. While this approach provides extensive customisability, it is foreign for the artists familiar with traditional digital art workflows. The UI offered by traditional drawing apps like Procreate [17] are specifically designed to suit manual creation via drawing. The Wand App attempts to mitigate this by replicating some UI design patterns from Procreate, but still requires users to learn proper AI Image Generation prompt technique [12].

Finally, all the AI systems in the existing solutions analyzed lack awareness of the user's intentions. They do not analyse the user's drawing to understand the user's desired subject which could lead to User intention-AI Image Model generation misalignment, especially in cases where the user prompt is brief. With no historical context awareness, users must repeatedly restate their intentions to the AI image generation model. In both cases, users must state their intentions clearly and explicitly to the image generation model which can become burdensome when users have to repeatedly interact with the AI Image Model to iterate, leading to a poor overall user experience.

1.2.2. Objectives

The objective of this project is to explore an alternative method of AI-assisted co-creation in the visual arts that is easy to use, iterative and aligned with existing artistic workflows. This is accomplished by designing and developing an AI-Assisted Drawing application: Broche that targets Apple's iPad mobile device and aligns well into existing artistic workflows like Procreate drawing app, but also introduces recent developments in AI image generation in the form of state of the art AI models to accelerate the artistic creation process.

1.2.3. Key Contributions

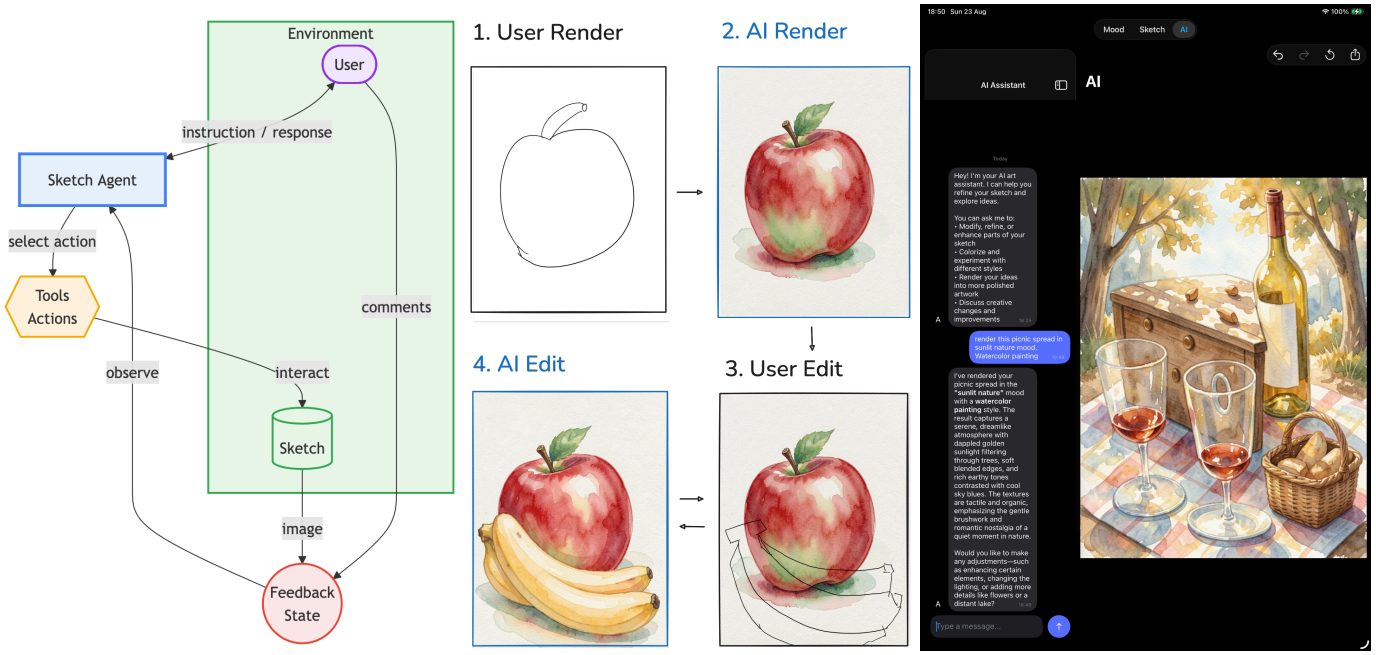


Figure 3. Agentic Architecture of Sketch Agent

Figure 4. Iterative AI Assisted Creation involves successive User-AI Interaction

Figure 5. Example of User Interface

The key contributions in this project are:

- **Agentic Architecture** This project implements a Sketch Agent architecture for interfacing with its AI features. This architecture is inspired by other successful applications of Large Language Models (LLM) to build LLM agents, such as WebGPT [18] and SWE Agent [19]. Users interact with the Sketch Agent, which is provided with access to various tools (actions) to accomplish the user's illustration (environment), such as a rendering tool powered by an Diffusion model. The user's comments and overlaid drawings are fed back to the Agent as feedback (state) to stimulate its next action
- **AI Deployment Models.** Both Offline and Online AI deployment models are extensively explored with tradeoffs compared to determine the optimal AI hosting model for the App. The primary hurdle encountered on Offline deployment is system resource constraint on Mobile Device (iPad). The main issue for Online deployments is API Key management and security. For the chosen deployment model (Online), mitigations are developed and discussed.
- **Iterative AI Assisted Creation** Persistence of conversation history from message to message allows users to discuss illustration ideas within the app itself and build upon previous conversational context. The project also implements direct image editing with a modern Diffusion model Flux.2 Klein 9B, allowing users to seamlessly iterate on an existing image without relying on inpainting masks [4].
- **User Interface Design and Prototyping** The project conceptualises and implements a User Interface for AI-Artist co-creation that is aligned with existing artistic creation workflows. For instance, a mood board feature is introduced to allow artists to gather inspiration in the app, while the sketch interface borrows UI design patterns from popular illustration app Procreate such as a layered sketch architecture and the 2 finger tap to undo gesture.

2. Literature Review

2.1. Advances in AI

2.1.1. Text Generation Models

LLMs can be traced from the original transformer paper which proposed modeling long form sequences only with attention. By replacing the status quo of Recurrent and Convolutional with Self-Attention layers on the model's own inputs, along with positional embedding to encode sequence position, the transformer architecture made training larger models possible due to its ability to effectively utilise modern hardware resources such as GPUs [20] . Liu et al. demonstrated the effectiveness of Decoder-Only Architectures for text generation by removing the Transformer's encoder when repurposing the model to generate text summaries [21]. OpenAI extended this idea by training a Large Decoder-only Language Model (LLM) with significantly more parameters [22] and fine-tuned it to follow human instructions [23], resulting in the early variant of ChatGPT known as InstructGPT.

2.1.2. Visual Language Models

Several advances in AI's ability to comprehend pictures have also been made in recent years. AI Text and Visual Encoder models convert text and images to vector embedding representations that more effectively encode semantic information for downstream tasks. CLIP demonstrated that it was possible to unify the Text and Visual Embeddings using a Linear mapping to a common embedding space [24], paving the way for image-text cross understanding in Visual Language Models (VLMs). Vision Transformers (ViT) show that it was possible for Transformer blocks to comprehend images as a series of small patches and positional encoding.

2.1.3. Diffusion Models

Diffusion is the key technology powering the current generation of AI Image Generation models. Inspired by principles of thermodynamics, Diffusion first converts an image to noise by progressively adding more and more random gaussian noise. A CNN model is then trained to reverse this noising process and reconstruct the original image from data. Trained models can then be used to generate images from random noise [25]. Rectified Flow augments the Diffusion process replacing the nondeterministic addition of stochastic noise with a direct linear mapping of data to noise, making the diffusion process more deterministic [26].

The Latent Diffusion paper subsequently introduced several improvements to make image generation with diffusion practical and useful. First, instead of performing diffusion in high dimensional pixel space, images were first encoded into a lower dimensional latent space, where diffusion was subsequently conducted. Latent Diffusion significantly reduced the computation cost of diffusion, which is instrumental for running on diffusion on consumer-grade mobile hardware. The paper also demonstrated the use of text prompt to condition the model's denoising process [27] with classifier free guidance where conditioned and unconditioned inputs are combined at a user tunable strength [28]. Users can now specify their desired subject with natural language.

Diffusion Transformers (DiT) pioneered the use of augmented ViT to replace existing CNN sections in Diffusion models with Transformer blocks to improve generation quality. DiT also introduced a computationally cheaper method of conditioning models in the form of Adaptive Layer Normalisation (AdaLN), where conditioning inputs are fed to to model directly in as the scaling and shifting parameters of Layer Normalisation layers [29]. Modern Image Generation models such as Qwen Image family [16] and Flux family of models are built on DiT blocks.

2.1.4. AI Model Selection

Heavy investment in AI coupled with competing multiple players in the AI industry has resulted in AI model fragmentation with a multitude of models to choose from. At the time of writing, Artificial Analysis's LLM Model Leaderboard lists 271 current generation model variants [30]. Each model family may also offer several variants with different parameter counts: [31] while each variant's weights can be quantised with many methods [32].

Online resources are available to assist with AI model selection. Artificial Analysis is an AI model benchmarking company that aggregates and compares the latest models across the industry [33]. LLM and VLMs are scored on output quality via weighted average of popular benchmarks [34] while Diffusion models are ranked based on an Elo rating calculated for human user provided votes on preference for output quality [35]. Together, Artificial Analysis aggregated information informs the model selection problem.

2.1.5. AI Agents

Recent research on LLM applications has increasingly focused on their use as autonomous AI Agents. Agents may perform actions by invoking tools that affect their environment to accomplish a given task and receive feedback about the environment's state in response to those actions [36]. They often have a memory component to remember the past state of their actions [37]. The effectiveness of these agents is highly dependent on the design of its allowed actions and the quality of feedback on their actions [19].

An early example of an LLM powered agent is WebGPT where a LLM is tasked with Question/Answering with the assistance of a web browser environment. The LLM was equipped with tools to interact in the environment, such as conduct searches, clink links or scroll the webpage to find answers. By allowing LLMs to find references on the Internet, the quality of the generated answers given greatly improved [18].

AI Agents have been successfully adapted to other domains. Most notably, Software Engineering AI Agents have been created to write code, with examples such Claude Code [38], Codex [39] and Antigravity [40]. A SWE AI Agent pairs a LLM with actions such as viewing, editing and searching source code files to resolve SWE tasks [19]. This demonstrates that AI agents can be adapted to solve problems in new domains.

Sub Agents are other specialised AI Agents that AI Agent may delegate specialised tasks to. The Sub Agent's primary advantage is Input Context Window Management: reducing the number of tokens fed as input to these LLMs. This is important as LLMs powering these agents have a hard cap on the number of tokens it can accept [41]. Empirical testing also demonstrates that LLMs struggle to pay sufficient attention to long inputs, even if they don't hit the token limit [42].The Sub Agents are given instructions in a fresh input context window, without interference from previous conversation history. Once complete, the Sub Agent summarises its actions and results before notifying the main agent. This allows the main agent to receive the relevant information while reducing the amount of additional context that must be maintained in its input window [43].

2.2. Software Engineering

2.2.1. Software Engineering Practices

Software Engineering (SWE) Practices provide standard workflows for common SWE tasks:

- **Requirements Analysis** is the process of eliciting both Functional Requirements that define the features the product must implement and Non Functional Requirements such as performance.
- **Reuse-Oriented SWE** is a development process where developer efforts in discovering and integrating existing components into their system instead of reengineering components from scratch to reduce overall engineering cost [44].
- **Design Patterns** are software architecture patterns designed to solve common problems encountered in SWE.
    - **Strategy Design Pattern** separates an implementation from its functionality through a common interface.
    - **Abstract Factory Pattern** decouples object creation from usage, allowing different implementations to be constructed at runtime.

By isolating implementation details from the core business logic; implementations can be replaced without affecting the underlying business logic [45].

2.2.2. iOS Development

Since the introduction of Swift in 2014 [46] as the lingua franca for development on Apple Platforms, the iOS development ecosystem, Apple's Mobile Platform, has been overhauled to orient the language. SwiftUI introduced a new declarative way of defining a UI [47], SwiftData added an Object Relational Mapping (ORM) that serialises Swift Model classes on disk [48]. Other useful Apple provided frameworks include CloudKit which allow applications to persist documents in the iCloud backed storage [49] and Pencilkit which provides useful UI primitives for building a drawing UI [50].

Reuse Oriented SWE can be applied in iOS Development by importing third packages using Swift's Package Manager to fetch prebuilt components for reuse. Exyte/Chat provides familiar text messaging UI components [51] while OpenAI [52] and Replicate Swift API clients [53] provide out of the box integration with external AI inference APIs.

2.2.3. Firebase

Firebase is platform-as-a-service offering from Google Cloud that offers prebuilt backend services including User Authentication and Document database (Firestore). It also offers hosting of custom backend services with Firebase Functions [54].Firebase runs on the same datacenter infrastructure that Google uses to host its products [55], making it a robust backend solution for small development teams with limited bandwidth to engineer the supporting backend stack from scratch.

2.2.4. AI Model Deployment

2.2.4.1. Online Deployment

As AI models have trended larger in recent years, performing inference on a server and exposing its function has been the preferred model for larger models that require the additional computing resources. Inference Providers like Replicate offer Inference-as-a-service, offering off-the-shelf API access to popular image generation models like Flux.2 Klein 4B [56]. These services allow AI experiences to be offered on mobile devices without being constrained by the limited system resources of such devices.

The proliferation of multiple inference providers for the same model made selecting a specific provider difficult. Model routing solutions like OpenRouter dispatches model inference API calls among multiple inference providers providing two distinct benefits: a reduction in development cost as the application only has to integrate with OpenRouter to access different models in a constantly evolving AI landscape, and fault tolerance, as the router automatically falls back to other providers, providing reliable access to models even as some providers experience downtime [57].

Securing the API keys sent to Inference Providers for authentication remains the key technical hurdle for the Online AI deployment model. Unintended disclosure of API Keys to AI Inference Providers such as OpenAI in source code [58] and Gemini API have resulted in significant bills of tens of thousands due to unauthorised usage of stolen API keys [59]. Untrusted users must be authenticated before they are given access to sensitive credentials. Some applications avoid the Key Management key problem altogether, instead instructing the user to Bring Your Own Key (BYOK), asking the user to input their own API keys to the AI Inference Provider [60]. However this requires that the user be technical and well versed in handling sensitive credentials.

2.2.4.2. Offline Deployment

Small Language Models (SLM) are smaller and or quantised variants of LLMs designed to run on local, resource constrained environments such as user's Laptop or Mobile Device. For instance, the Qwen 3 0.6B model contains only a fraction of the weights of the full Qwen 3 model, achieving a significant reduction in compute & memory requirements [31]. Further quantisation of model of model weights lower precision types FP8, Int8 or even Int4 further reduce the system resources necessary to utilise such models. However, a reduction on model parameters or weight precision typically translates to loss in model performance.

Renewed interest in on-device AI is evidenced by the emergence of several on-device Inference Runtimes for serving SLMs on iOS devices. Apple's Foundation Model APIs provide access to an on-device SLM used to power iOS's Apple Intelligence features in recent iOS versions [61]. Apple's MLX is a tensor array framework in the class of Pytorch or Tensorflow with native acceleration on Apple Silicon M class processors [62]. MLX-Swift-LM builds upon this foundation to run SLMs on M series processors [63]. Third party Inference runtimes for iOS devices also exist, including the Tensorflow based LiteRM-LM [64] and UZU [65] , both relying on low level Metal Hardware Shaders to achieve hardware acceleration on Apple's M series processors.

In addition to SLMs, efforts have also been made to port smaller and/or quantised Diffusion models on iOS devices. Apple seminal work on porting Stable Diffusion model to the iOS platform (coreml-stable-diffusion) [66] demonstrated that it was possible for these models to run on resource constrained mobile platforms. Third party developers have stepped in to fill in the gap for modern model variants, such as Media Generation Kit's [67] support for Flux.2 model variants.

2.3. Product Design

2.3.1. End User Workflow

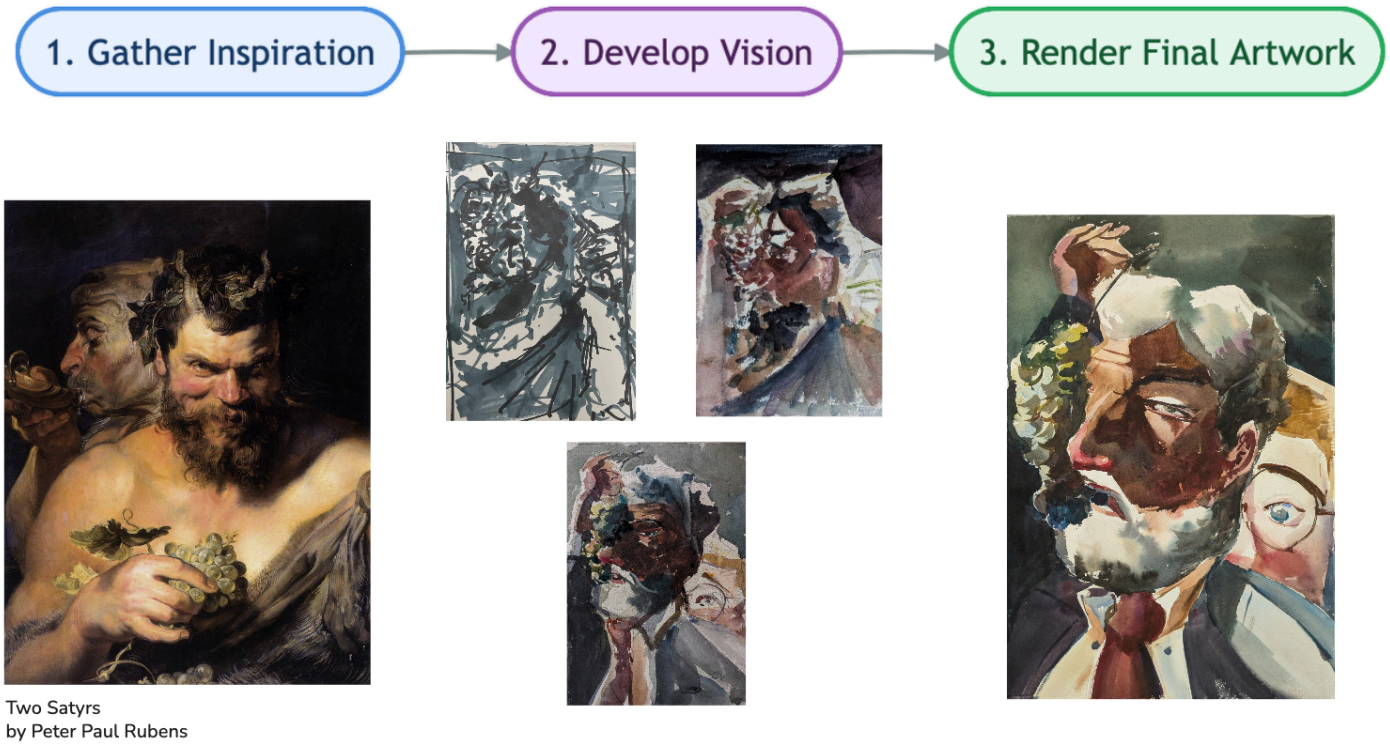


Figure 6. Traditional Media Artist Workflow. Work Samples from [68].

While exact specifics of artistic workflow may vary from artist to artist, artists in general begin their creative process by first gathering inspiration by crafting a mood board of images on online platforms such as Pinterest [69] to gather aesthetic elements they might reproduce in their final piece [70]. For instance, they might be inspired by the color scheme of one artist's work while being captivated by the placement of subjects in another photograph. They then explore the composition, values and color of the final piece by developing several rough sketches [71]. Finally, they render the final artwork based on artistic direction they have made prior explorations [15].

2.3.2. Competitor Analysis

An analysis of prior art was conducted to identify a gap in the market of existing solutions:

- **ComfyUI** expresses AI image generation as Directed Acyclic Graph of processing steps, offering extensive customisability of each step from model weight loading to Distillation sampler tuning [8].
- **Draw Things** provides a friendly mobile-first UI around these Diffusion models, allowing non technical artists to quickly load and prompt these models significant technical expertise on their own Apple devices [9].
- **Draw Anything** sidesteps the limitations of text prompting by allowing the user to craft an “image prompt” that is passed the AI model to define overall composition and placement of the final subject [10].
- **Sagebrush** extends this concept closer to artistic style by allowing this initial sketch to be drawn using an iPad and Apple Pencil that they are likely to already have for digital artistic creation. User-created and AI-generated images are placed side by side for evaluation.[11].
- **Wand App** also further integrates into the artistic workflow by borrowing UI elements from popular illustration app Procreate such as layers. It offers regional editing capabilities by using AI models to replace a section of an image delineated by a user-specified mask, known as Inpainting [12].

3. Analysis and Design Approach

3.1. Product Design

Figure 7. UI Design Influences

The objective of Product Design is to derive suitable UI design for the Broche App that is aligned with existing Artist's Workflows. Multiple UI wireframes were developed to explore the practicality of various UI elements and layouts.

- **Mood Board.** Inspired by pin boards artists build on sites like Pinterest to gather inspiration, the App introduces a similar interface for collating images and textual descriptions that depicts the moods, themes or color schemes they intend to capture in the final piece.
- **Sketch View** The main drawing interface allows the user to sketch out the general composition of their sketch as a visual prompt for Image Generation or to mark out areas for amendments in image editing. The user is provided a toolbar of drawing tools to choose from.
- **AI Assistance** Access to the embedded Sketch Agent is provided via text messaging interface, placed side-by-side to the current Sketch. This allows the user to immediately review and give feedback to the Sketch Agent's changes during interactions.

3.2. Software Architecture

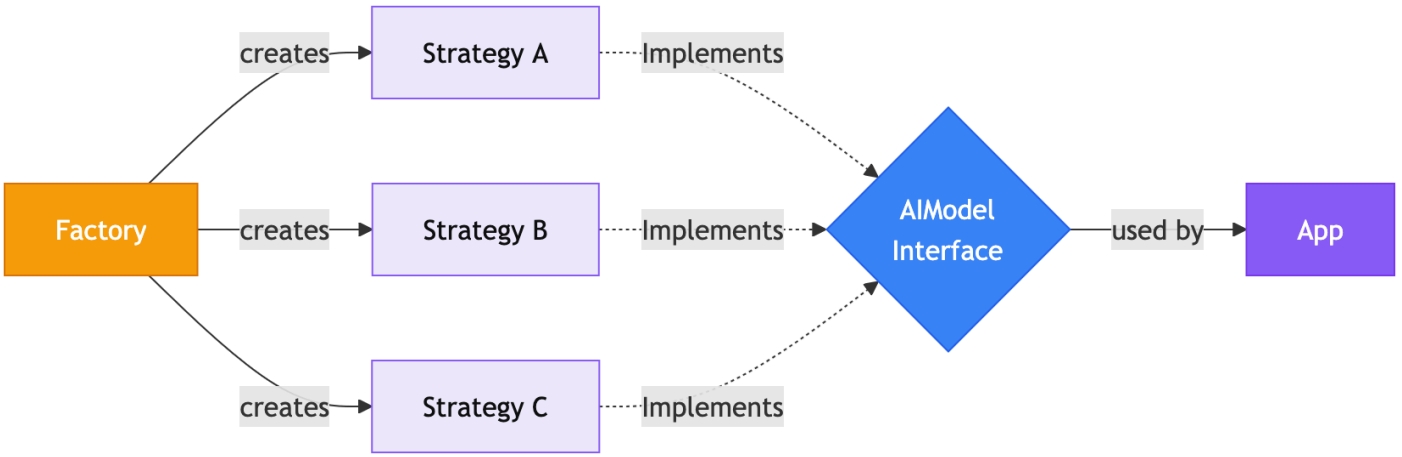


Figure 8. AIModel Interface featuring Strategy and Factory Pattern

The Strategy Design Pattern was employed to segregate AI Model Interactions from the rest of the App. Instead of depending on specific implementations directly, Broche's core logic depends upon the AIModel Interfaces for interacting with Text, Visual-Text and Image AI models. The Factory Design Pattern was employed to create the concrete implementations of the AI Model interface.This design provided 2 key benefits:

- **Isolate Changes** Due to initial uncertainty on specific AIModel Implementation that would be most suitable for the Broche App, use of the Strategy allowed the experimental evaluation of multiple AI Model Deployment methods. This allowed multiple implementations of the AIModel to be tested and replaced without impacting the code in other areas of the App.
- **Robust Testing** Mocked AI model implementations could also be injected to isolate application code for testing, without inference from nondeterministic AI models. This can be done by swapping the default AIModel Factory with the mock AIModel factory which created AIModel instances that simulated AI models with hardcoded outputs.

3.3. AI Model Selection

AI Model selection has been performed carefully to ensure the right model-product fit, considering factors such as output quality, cost and output latency. The Sketch Agent is powered by 3 types of AI models: LLM's for text generation, VLM's for visual prompted text generation and a Image/Text-to-Image Generation model for image generation. Model options are collected from the Artificial Analysis list of current models for each category required by Agent: LLM, VLM, and Image Generation Models.

Figure 9. Low Quality Image Output of Stable Diffusion 2.1, 6 bit Quantisation

Some AI Model variants were cheap to run, but produced poor quality output. The 6 bit quantised Stable Diffusion v2.1 model was found to generate substandard images while the 0.6B parameter Qwen 3 model was determined to be unable to follow clear instructions. Manual evaluation of model output quality was performed to weed out underperforming models.

3.4. AI Model Deployment

3.4.1. Offline Deployment Architecture

Running AI models locally on-device offers significant robustness, privacy, and cost benefits. Offline models do not require an internet connection to operate, while keeping user data entirely on-device. This ensures that user data is not collected or reused by model providers without the end user's consent. In addition, on-device model inference incurs no per-request inference costs.

Based on these perceived advantages, initial exploration focused on offline AI model deployment architectures, with particular emphasis on offline execution, data privacy, and zero-cost inference. For LLM and VLM models that generate text, the MLX, UZU, and LiteRT-LM inference runtimes were evaluated. For diffusion models, Apple's Core ML Stable Diffusion and Media Generation Kit were explored.

However, several limitations hampered the implementation of an offline, on-device AI model inference architecture in the App:

- **Model Limitations**
    - Although UZU was identified as the best performing inference runtime in our benchmark, its limited model support does not extend to VLM, which the App requires for user drawing captioning and Mood description generation.
    - coreml-stable-diffusion was able to generate images using a 6 bit quantised variant of the Stable Diffusion 2.1 model but its image generation quality of the model was insufficient. It does not support newer models with improved generation quality.
- **Unsupported Features.** Although Media Generation Kit was able to perform image generation with the new Flux.2 model with a text prompt, it was unable to do so with an image prompt, causing a runtime crash. Image prompted generation is required by the App to supply the user's drawing as a visual prompt for Image Generation.
- **Slow Model Load** Due to iOS App bundle size restriction [72] of 4GB, AI Model weights have to be distributed separately from the App bundle. Model weight sizes are often large, resulting in poor user experience where the user has to wait for a slow download to complete before they are able to use the AI features on the application. It also takes a few minutes for most inference engines to load the model before it is ready.

- **High Resource Utilisation** Performing Model inference on-device places a significant strain on the system resources of the resource constrained iPad. In particular, models loaded on the device are constrained by within 8GB unified memory that the iPad has. Just loading the 4 Billion parameter Qwen 3.5 VLM model consumes 5.2GB of the 8GB of iPad memory.
- **Weaker Models and Smaller Context** Limited System Resources force the use of weaker models that may not achieve the desired performance and disqualifies the loading of more than one model at a time. Since memory consumption also rises linearly with the model's input context window size, a hard cap is imposed on the size of input context which the model bases its outputs on.
- **Slow Builds and Dependency Conflicts** Running Models on device means bundling all dependencies necessary to conduct model inference together with the rest of the App. Complex dependency chains take a long time to build and create more opportunity for conflicts. For example, MLX and Media Generation Kit could not be built together because they both exported conflicting symbol names.

3.4.1.1. Online Deployment Architecture

The Online Deployment architecture rectifies many of the limitations of the Offline Deployment architecture, with access to a wide array of state of the art models, no slow model download or load times, minimal resource utilisation and no dependency bloat However, there are trade-offs when shifting to the Online Model. Model inference performed by external inference providers incurs cost. Secure management of API Keys to inference becomes a core issue.

Directly embedding API Keys in the source code is an invitation for bad actors to extract the code [58]. Several System Architectures were explored to mitigate the risk of unintentional key disclosure:

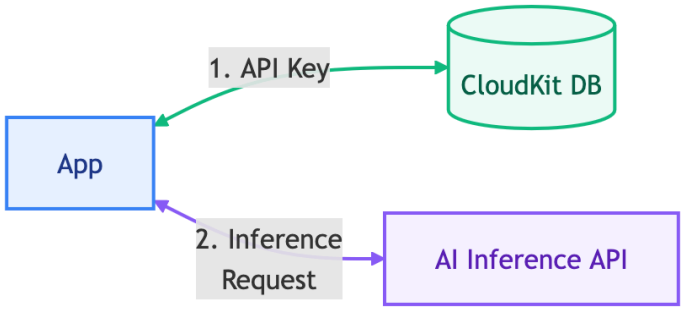


Figure 10. CloudKit Database Architecture

- **CloudKit Database** This architecture offloads the API Key to the public CloudKit Database. Before making an Inference Request to an AI Inference API, it first fetches the long-lived API Key from the Database, then contacts the Inference Provider's API directly to make the AI Inference Request.

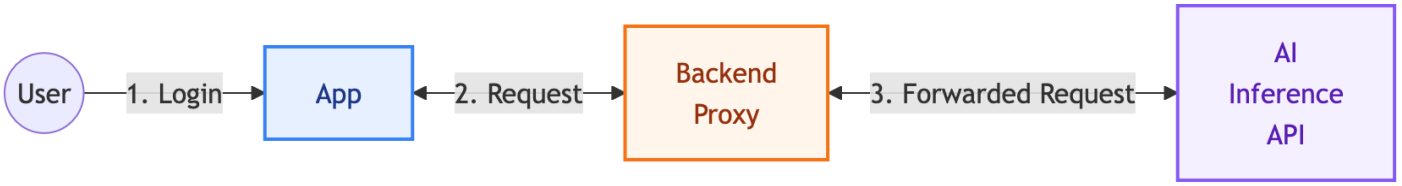


Figure 11. Authenticated User with Proxy Architecture

- **Authenticated User with Proxy** This design restricts access to authenticated users only. Using Firebase Authentication, users must authenticate before using the Broche App. The API Key is never

disclosed to the client App. Instead, the App may only make requests indirectly via a backend proxy server, which is responsible for relaying the Inference request to the actual Inference Provider.

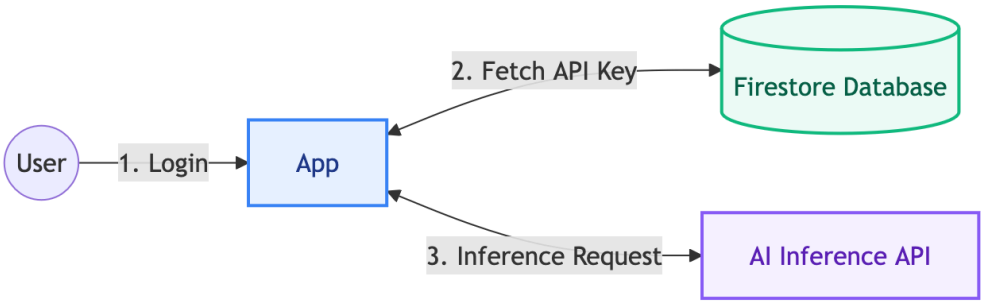


Figure 12. Authenticated User with Database Architecture

- **Authenticated User with Database** Again, users are forced to authenticate before using AI features. Authenticated users are given direct access to the Inference API by fetching a record containing the long-lived API Key in the Firestore Database.

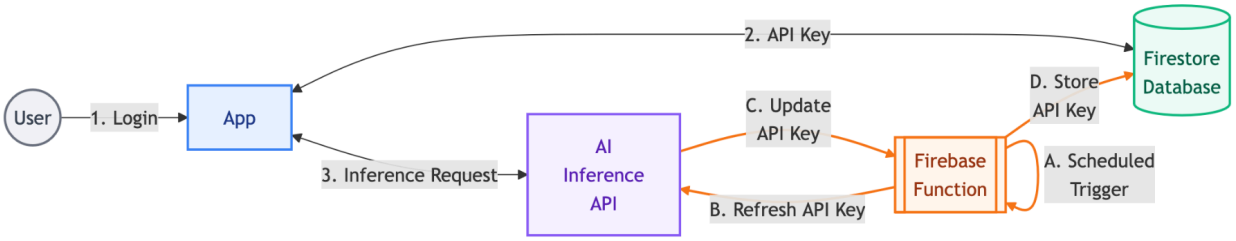


Figure 13. Authenticated User with Database and Key Rotation

- **Authenticated User with Database and Key Rotation** Extends the design by the addition of Firebase Function that periodically rotates API keys stored in the Firestore Database. The job contacts the AI Inference providers to issue fresh API keys and subsequently writes to the Firestore Database. The App obtains the rotated API Keys from the database prior to making a request to an inference provider, ensuring that it always uses the latest API key for authentication.

Table 1. Evaluation of System Architectures for API Key Security

Architecture	Security	Complexity	Cost	Latency
Cloud Kit Database	**Weak**. Any user with access to the App and the iCloud account can access the API Key in the CloudKit public database and utilise its AI features.	**Low**. Simple to implement with CloudKit API available in the iOS SDKs [49].	**Low**. CloudKit costs are bundled together with the Apple Developer Membership [49].	**Fast** Database lookup for API key.
Authenticated User with Proxy	**Strong:.**Inference Provider API Keys are never disclosed to the App. Only Authenticated Users are able to utilise its AI features	**High**. Requires the implementation of a custom Backend Service to proxy requests to AI Inference providers.	**High**. Network Egress charge is incurred for all proxied data between the App and Inference Provider [73].	**Slow** Proxied requests have to traverse multiple network hops.
Authenticated User with Database	**Moderate**. Only Authenticated users can fetch API Key from the Database and access AI Features. However a long-lived API key may still be disclosed if the App is compromised.	**Low**. Simple to implement with Firebase SDK.	**Low.** Since the API key is long-lived, App clients can cache the fetched API key after first reading it from the API server.	**Fast** Database lookup for API key.
Authenticated User with Database and Key Rotation	**Moderately Strong.** Restricts access to API Key and AI features to authenticated users. Short-lived, frequently rotated API keys reduce the potential damage should the API key be disclosed by a compromised App.	**Medium.** A scheduled Firebase Function has been implemented to automatically rotate API keys and write the result to Firestore.	**Medium** Database reads to fetch API Keys are relatively cheap [73].	**Fast** Database lookup for API key.

3.5. Sketch Agent

The Sketch Agent is an AI Agent that is designed by the user in their Sketch (Drawing) by acting as an conversational entry point to all other AI-Assisted features of the App. It is designed to assist users in two key workflows:

- **Editing Workflow** This workflow is used when the user wants to modify, refine, or otherwise make a relatively small change to their Sketch.
- **Rendering Workflow** This workflow is used when the user wants to render their rough drawings into finished pieces, significantly transforming their Sketch.

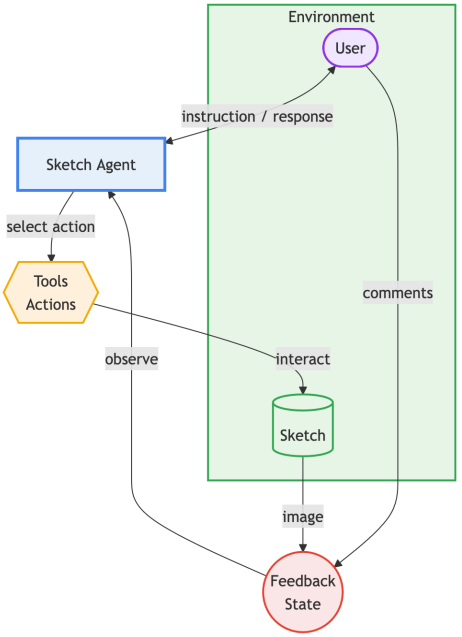


Figure 14. Sketch Agent Interactions with Sketch Environment

The Sketch Agent is agentic in that it takes in stimulus from the environment composed of the user's Sketch and messages, decides on course of action and takes action by invoking tool calls. These tool calls alter the environment by making amendments on the user's Sketch. The Agent receives feedback on its actions from the environment and uses this as stimulus for its next action if needed.

The Agent is designed to accomplish its task with the following tools:

- `caption_sketch` Since the Agent is unable to directly perceive the user's drawing, this tool provides the agent with a text description of the current subject of the user's current sketch, informing about what the user has drawn / intends to draw.
- `explain_edit` Specialised variant of the caption_sketch tool that helps the Agent understand the purpose and intent of the user's edit from their drawing strokes.
- `edit_sketch` Allows the Agent to apply an incremental edit to the Sketch as per user's instructions.
- `render_sketch` Allows the agent to transform the user's drawing into a polished final piece as per user's instructions.
- `list_mood` Lists all user-created moods styles that the user has created in their mood board.
- `inject_mood` Retrieves the description provided for a specific mood that the user has specified. This helps the agent understand the intricacies of the specific mood and align future actions along this mood.

4. Detailed Implementation

This section discusses the implementation of the Broche App. Its source code is available at https://github.com/mrzzy/ntu-fyp.git.

4.1. UI Prototype

Subsequently a full UI prototype was developed with SwiftUI based on the most promising UI layout explored in UI wireframes.

4.1.1. Mood Board UI

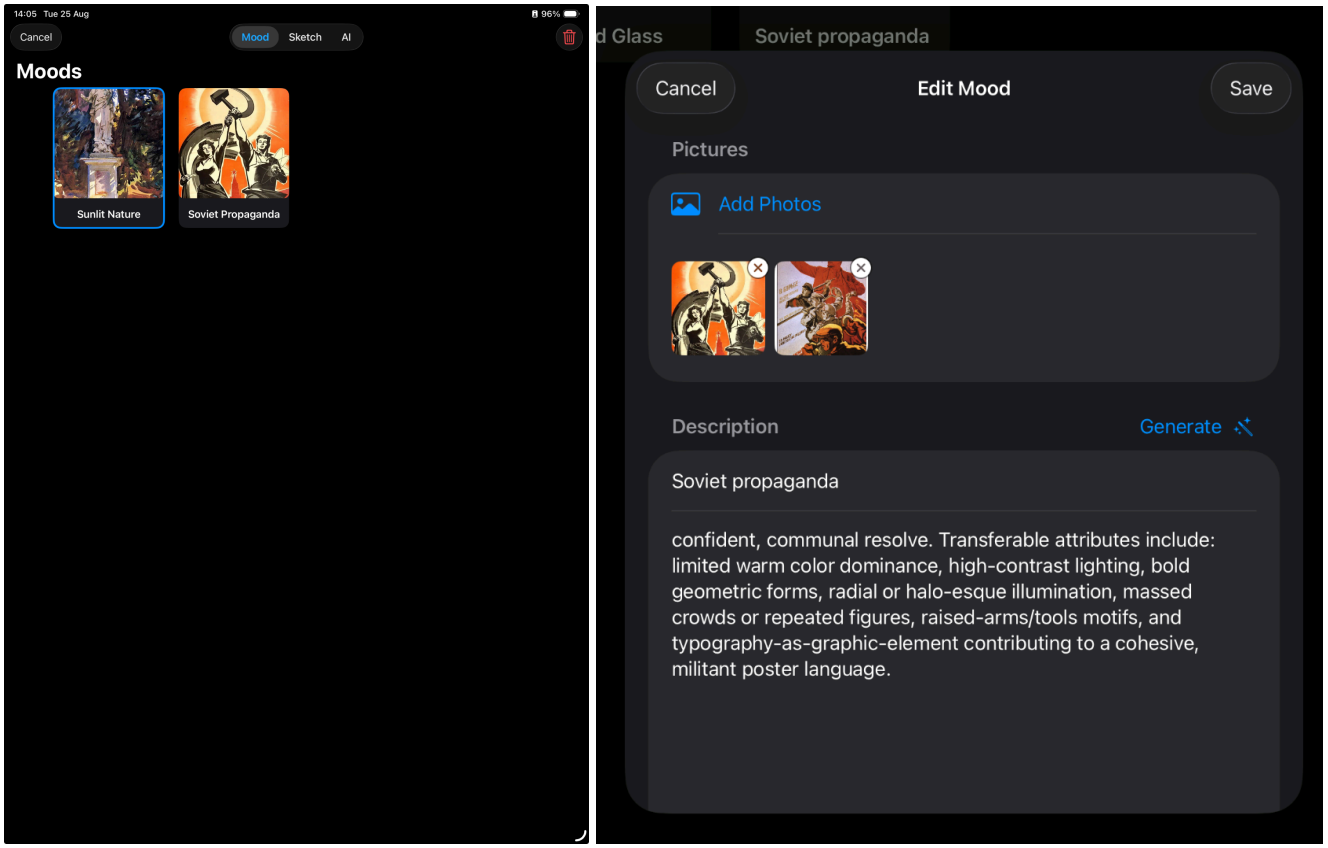


Figure 15. Mood Board UI featuring Select-and-Delete Gesture
Figure 16. Mood Editing UI featuring AI Generated Description

The Mood Board UI provides a space for users to collect the desired Mood and Themes in Grid Layout similar to Pinterest's UI Layout [69].

- **Mood Management** The "+" toolbar button allows users to add moods, while a Long Press gesture allows users to select and delete Moods.
- **Photo Selection** integrates with Apple's PhotoKit to select photos from their photo library when creating moods.
- **Manual / Auto Description** Users may manually type a description of their mood or they may automatically quickly generate a description from the attached photographs using an integrated VLM model.

4.1.2. Sketch View UI

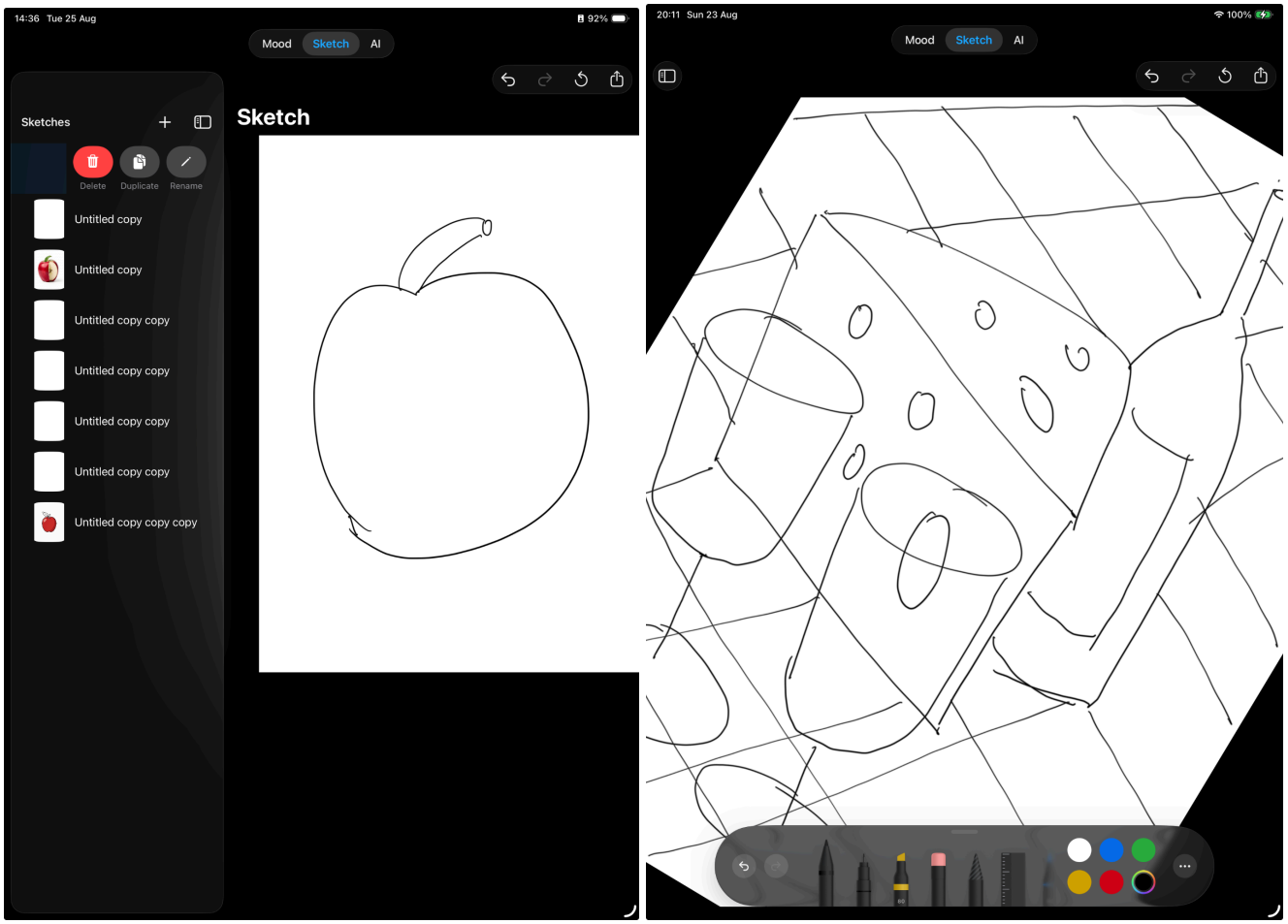


Figure 17. Sketch View Featuring Management SIdebar Swipe Actions
Figure 18: Drawing Toolbar and Zoom and Rotation of Sketch View UI

The drawing interface was implemented with Apple's PencilKit providing drawing functionality using the Apple Pencil stylus, with a floating toolbar for Drawing Tool and color selection. Inspired by the traditional drawing app Procreate, it borrows several UI design patterns to ensure that the UI experience remains familiar to

- **Drawing Selection and Management.** A sidebar implemented as a Navigation Split View allows users to create & select the specific drawing they intend to work on. Swipe actions provide additional functionality to copy, rename or delete drawings.
- **Free Zoom and Panning** The PencilKit drawing canvas view is wrapped in a customized scroll view to implement two finger gesture driven free zoom and rotation pan of the drawing canvas like Procreate. A reset button is provided to reset the drawing in the original orientation.
- **Undo and Redo** Users can undo or redo changes to the Drawing with Redo / Undo buttons. Procreate's Two Finger Tap Undo and Three Finger Tap Redo gestures are also implemented.

4.1.3. AI Assistance UI

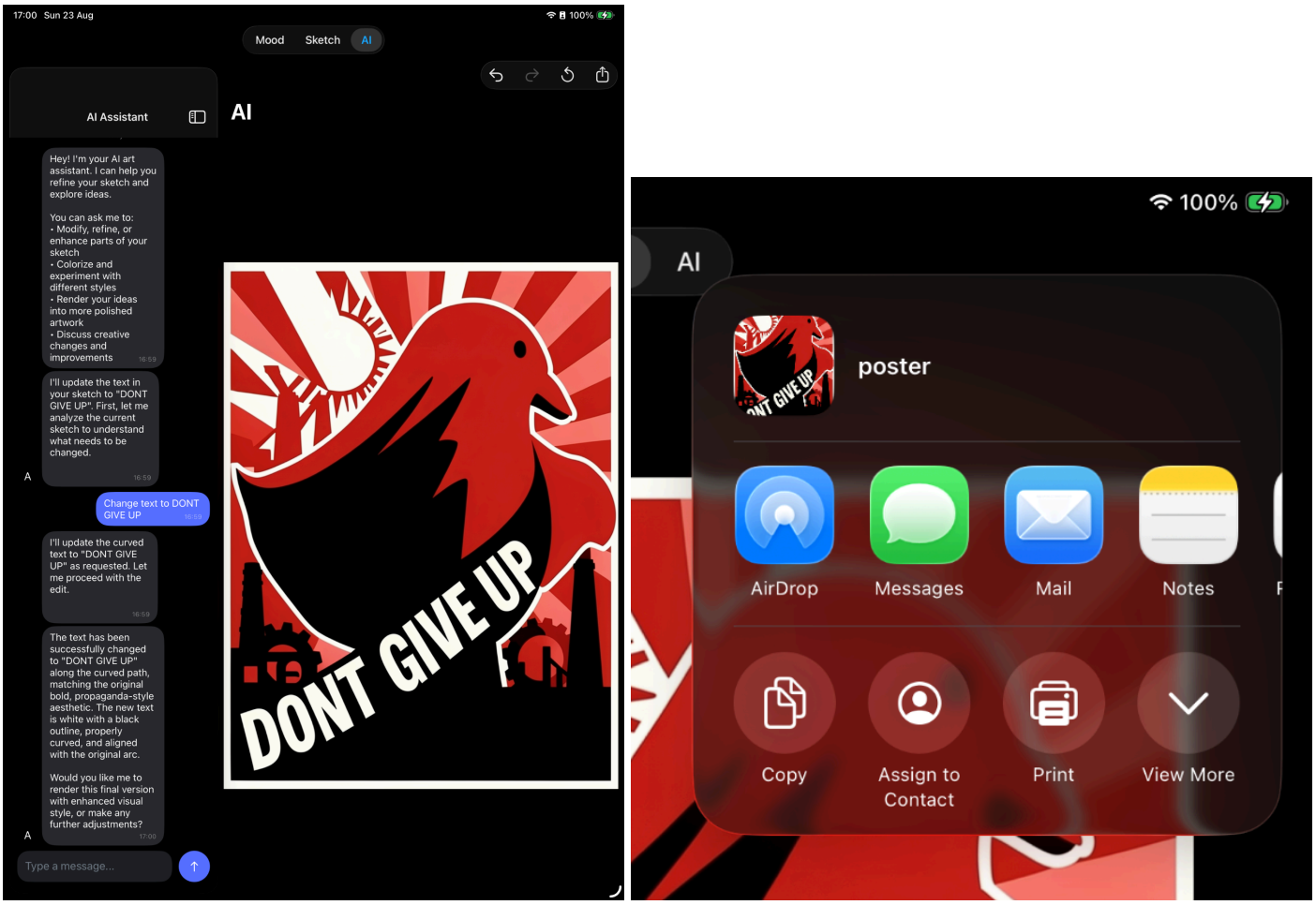


Figure 19. AI Assistance UI featuring Side by Side Text Messaging and Sketch

Figure 20: Drawing Export UI featuring popup with Multiple Sharing options

The AI Assistance UI depicts a text message for interacting with the Sketch Agent side-by-side with the current drawing for reference.

- **Side by Side** The Navigation Split View originally intended for sidebar navigation was repurposed to host both Text Messaging, and Drawing Preview.
- **Text Messaging View** Exyte/Chat's text messaging UI components were brought in to implement the messaging interface with a minor tweak to fix a message rendering issue in the non-full screen layouts [74].. The familiar interface of text messaging apps similar to Telegram/WhatsApp provides an inviting gateway for users unfamiliar to new AI features such as image generation.
- **Undo and Redo** Users can undo / redo AI-generated modifications to their drawing just like any user-created change. This allows them to iterate on AI-generated changes by adjusting their prompts to the Sketch Agent until they achieve the desired results.
- **PNG Export** A completed drawing can be shared as a PNG image using the Share toolbar button which displays a sharing dialog that presents the user with various export options.

4.2. Data Modeling & Persistence

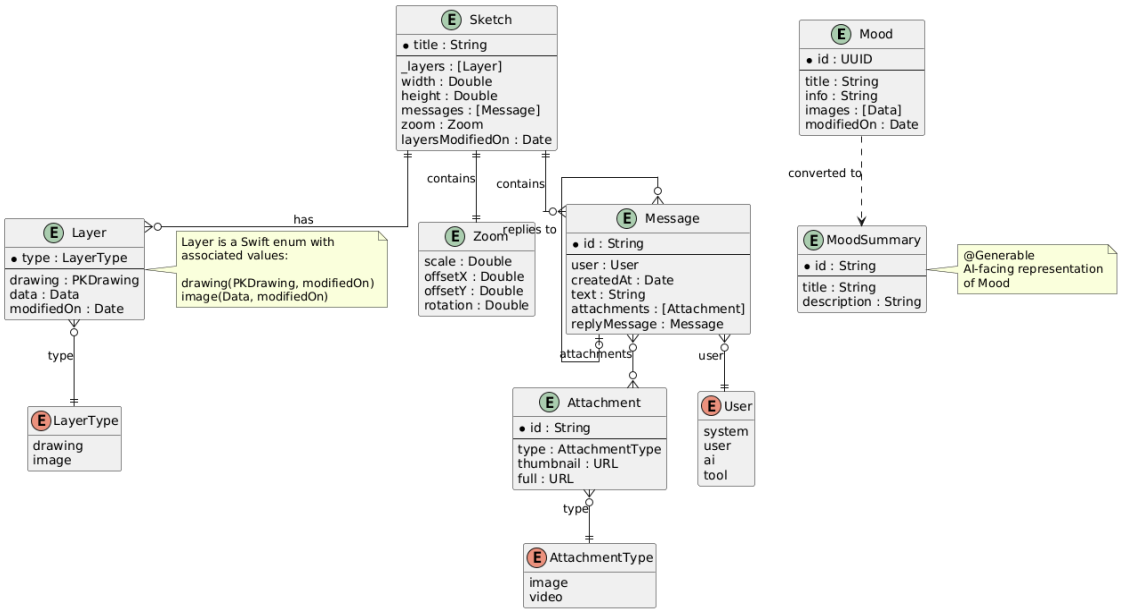


Figure 21. Data Model Entity Relation Diagram

Data was modeled as Swift Classes and persisted across application restarts using the SwiftData ORM.

- **Denormalised Model** Early in Development, the Message Entity was modeled as a separate Swift Data Model class and associated with the Sketch Model via a foreign key relationship. However, that resulted in App crashes due to referential integrity violations when the Sketch model was deleted while Message model is reused elsewhere in the App. To resolve the issue, the message was denormalised and stored as an embedded field within the Sketch model.
- **Layered Architecture** Drawings are encoded as a stack of overlapping layers of user drawings and AI images. This non-destructive architecture allows changes to be easily undone by unwinding the stack of layers. SwiftData's natural integration of model changes into the system's undo manager allows the addition of layers to be easily reversed by the undo manager. The design was inspired by the existing Wand Drawing App [12] and Checkpoint and Revert feature in Software Coding Agents like SWE agent [19].

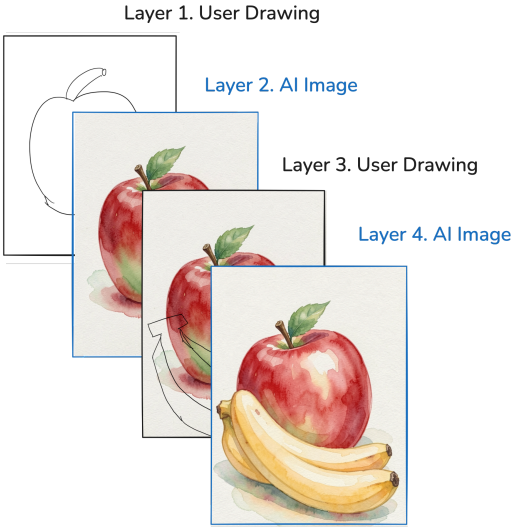
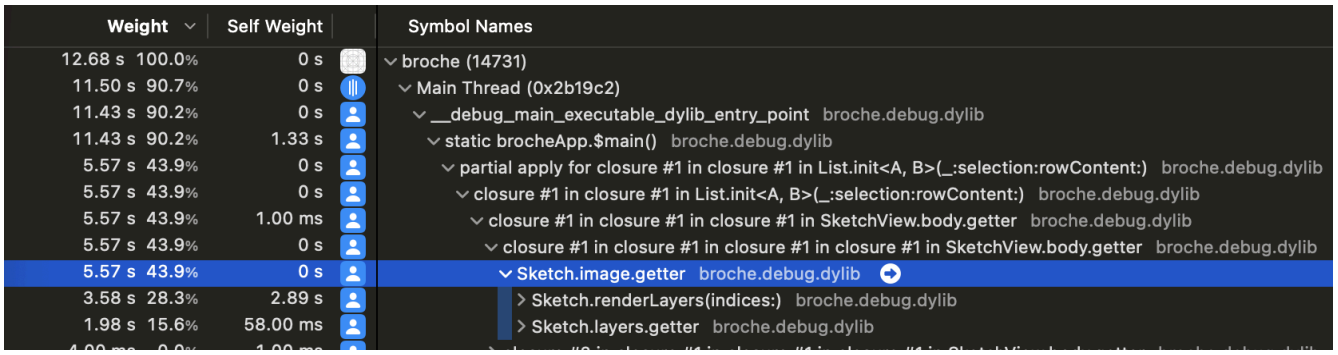


Figure 22. Sketch Model's Layered Architecture facilitates Undo

4.3. Performance Optimisation

Acceptance Testing of the earlier version of App revealed that the Sketch View UI identified 2 performance issues that significantly impaired the user's experience on the Broche App for two workloads:

1. **Drawing Stroke** Each drawing stroke took several seconds for the app to register and render.
2. **Panning and Zooming** Panning and Zoom the Sketch View also exhibited a noticeable lag of a few seconds before the view reflected the gesture for drawings with an AI generated image layer inserted.



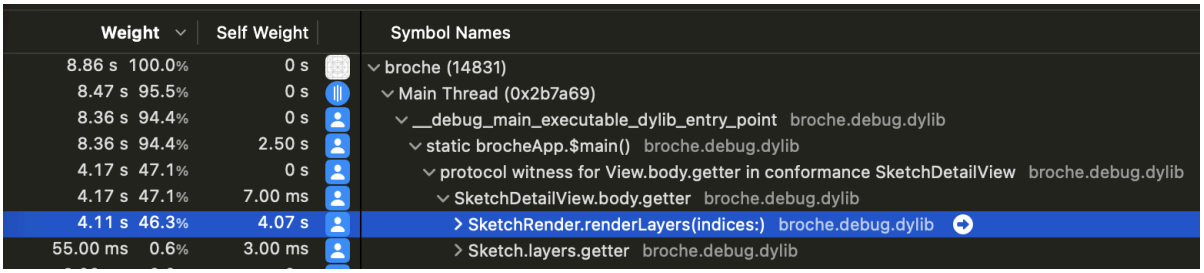


Figure 23. Call Graph for "Drawing Stroke" Issue.
Figure 24. Call Graph with Time Profile for "Panning and Zooming" Issue

With Instruments, the built-in profiler in Apple's Xcode Integrated Development Environment, a sampling based Time Profiler was employed to collect call samples to identify the Root cause of the performance bottleneck.

- **Drawing Stroke** Analysing the call graph, one can observe that a significant portion of execution time is spent on the Sketch model's image rendering pipeline (`Sketch.image.getter`). These calls are made by the List view to render image previews of the drawings on the side-by-side selection sidebar. Since SwiftUI redraws the entire sidebar every time as part of its functional design, the Sketch image rendering pipeline is invoked on each view refresh.
- **Panning and Zooming** When panning and zooming, Sketch rendering (`SketchRender.renderLayers`) takes up the majority of all execution time. As per SwiftUI's design, each layer of the Drawing is re-rendered and combined into a single image whenever the user Pans or Zooms the drawing, resulting in a user perceivable delay.

Once the bottlenecks have been identified, optimisations are applied to resolve the performance issue.

- **Drawing Stroke** Redrawing the drawing image from scratch on every refresh is unnecessary. Caching has been introduced to store the rendered drawing image and avoid redundant rendering. The cached image expires after 5 seconds, triggering a new image render.
- **Panning and Zooming.** Making the observation that only the top layer is subject to significant modification by the user when drawing. Constantly re-rendering of the other layers of the drawing is redundant. The optimised rendering path instead only renders the bottom layers once as a cached image, only constantly rerendering the top layer to be responsive to user input.

Table 2. Performance Optimisation Results

Optimisation	Benchmark	Baseline	Optimised	Speedup
Drawing Stroke	1000 `Sketch.image.getter` calls	21.21 s (21.21 ms/get)	41.74 ms (41.74 µs/get)	508.18×
Panning & Zooming	1000 3-layer (Drawing, Image, Drawing) Sketch Renders	26.88 s (26.88 ms/render)	5.86 s (5.86 ms/render)	4.58×

4.4. AI Model Selection

Implementation of the Sketch Agent involved selecting models in 3 categories:

- **LLM** As core model powering the Sketch Agent, the LLM converses with the user and dispatches tool calls.
- **VLM** Complements the Sketch Agent's LLM where visual comprehension of images is required.
- **Diffusion** Image Generation model used by Sketch Agent to generate edited or transformed versions of the user's sketch.

Models with lower inference cost were prioritised to keep costs low, subject to a meeting minimum generation quality requirements. Although single VLM can serve both LLM and VLM roles, the models were chosen separately to reduce cost. VLM models are more expensive than LLM models, so they are only used when necessary.

Table 3. AI Model Selection

Category	Model	Rationale	Cost (21 Aug 2026)
LLM	**Qwen3-30B-A3B-Instruct-2507**	Mixture-of-Experts (MoE) model with strong instruction-following capabilities. Built in support for making tool calls.	**Input:** $0.04815 / 1M tokens **Output:** $0.1931 / 1M tokens [75]
VLM	**GPT-5 Nano**	Multimodal VLM with visual comprehension capabilities for analysing user drawn content.	**Input:** $0.05 / 1M tokens **Output:** $0.40 / 1M tokens [76]
Diffusion	**FLUX.2 Klein 9B**	The 9B parameter variant of Flux.2 model balances image quality and speed. Step-distillation was applied on the model for faster image generation in just 4 diffusion steps [4].	**Input:** $0.002 / image megapixel **Output:** $0.015 / image megapixel [77]

4.5. AI Model Deployment

Based on the evaluation of Offline and Online AI Model Deployment Architectures, the Online Architecture was chosen as it was more suitable the Broche App:

- **Higher Resource Limits** Afforded by the Online Architecture to make it possible to run large models like Flux.2 Klein 9B and multi-model designs.
- **Warm Immediate Inference** AI Model Inference can be conducted immediately without having for AI Model weights to be downloaded and loaded.

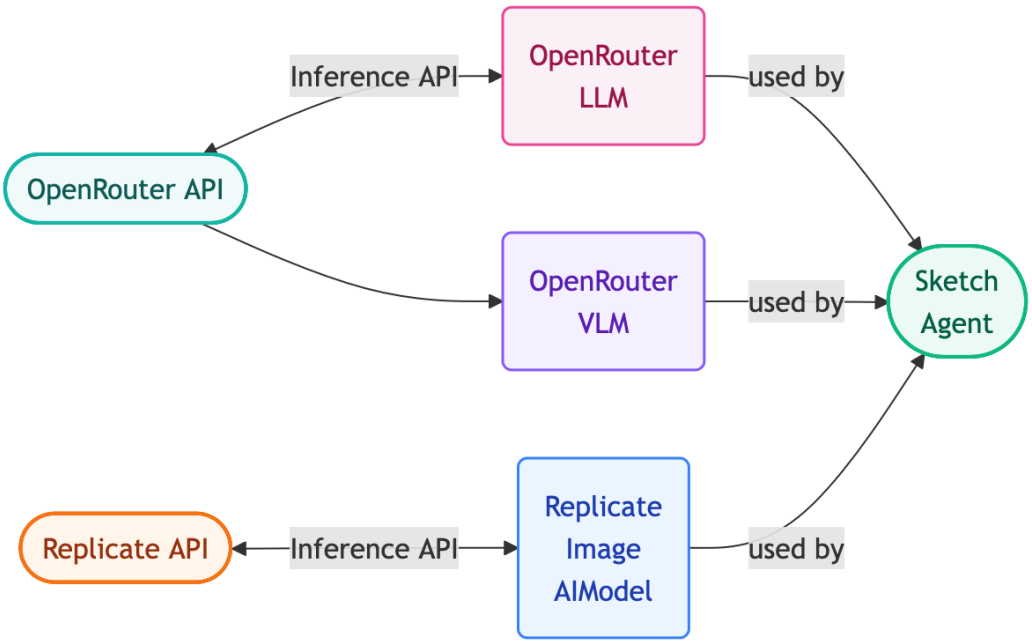


Figure 25. Online AI Model Deployment Architecture

The Broche App integrates with 2 Inference Providers to serve AI Model Inference requests.

- **OpenRouter** Integration with the OpenRouter was added to serve LLM and VLM requests using the OpenAI Swift SDK. This is possible as OpenRouter exposes a OpenAI API-compatible endpoint [78].
- **Replicate** Integration with Replicate was added to serve Diffusion model requests using the Replicate Swift SDK.

To safeguard API keys against disclosure, the App adapts the Authenticated User with Database and Key Rotation system design. A scheduled Firebase Function is triggered regularly to create new API Keys with Inference Providers and delete the old keys. The new keys are then saved into the Firestore Database at predetermined locations for easy retrieval by the App for authenticating Model Inference requests to the Inference Providers.

4.6. Sketch Agent

At its core, the Sketch Agent is powered by Qwen3-30B-A3B-Instruct-2507 LLM that comprehends and responds to user messages, determines its course of action and dispatches tool calls to execute its planned actions. The available tools are declared to the LLM as a series of function declarations with specific input and output schemas.

The LLM's behaviour is guided by the Sketch Agent System Prompt that provides guidance on its intended behaviour. Without instructions from the system prompt, the Sketch Agent's ability to correctly call tools is

unreliable. For each workflow, the System Prompt instructs the LLM on the specific tool calls it should invoke.

4.6.1. Agent Workflows

Editing Workflow The Editing Workflow is invoked by the agent when the user wants to modify, refine, or otherwise edit an existing sketch.

1. The agent first uses `ExplainEditTool` to interpret the user's intent from their drawing and messages into an actionable edit description.
2. If the user specifies a particular mood or style, the agent calls `ListMoodTool` to identify the available moods.
3. It then uses `InjectMoodTool` to retrieve the full description associated with the selected mood.
4. Finally, the agent invokes `EditTool` with the resulting edit instructions to produce the modified sketch.

Rendering Workflow The Rendering Workflow is invoked by the agent when the user wants to render their sketch into a polished image.

1. The agent first uses `CaptionTool` to generate a visual description of the user's drawing to understand the intended subject and composition behind their drawing.
2. If the user specifies a particular mood or style, the agent calls `ListMoodTool` to identify the available moods.
3. It then uses `InjectMoodTool` to retrieve the full description associated with the selected mood.
4. Finally, the agent invokes `RenderTool` with the rendering prompt to produce the final polished image.

4.6.2. Agent Tools

Tools are implemented using Apple's FoundationModels API to leverage its JSON schema generation and JSON output parsing facilities. FoundationModel's Generable has the advantage of relaxed syntax parsing, meaning incomplete JSON outputs from the AI Model would not cause parsing to fail.

Detailed explanations of how the Sketch Agent interacts with each tool:

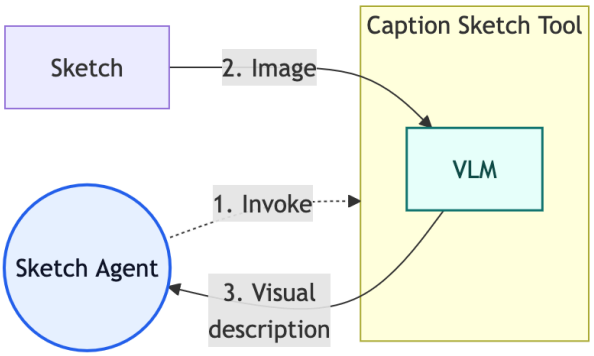


Figure 26. Caption Sketch Tool

- **`caption_sketch`** Analyses the current drawing using a GPT-5 Nano VLM Sub Agent. The Sketch Agent may provide an optional hint to aid in the sub agent in its captioning which useful situations

where the user has explicitly indicated their intended subject. The VLM Sub Agent is instructed to infer the user's intended subject from an image of the user's Sketch. This text description is returned back to the Parent Sketch Agent to provide better understanding of the user's current sketch.

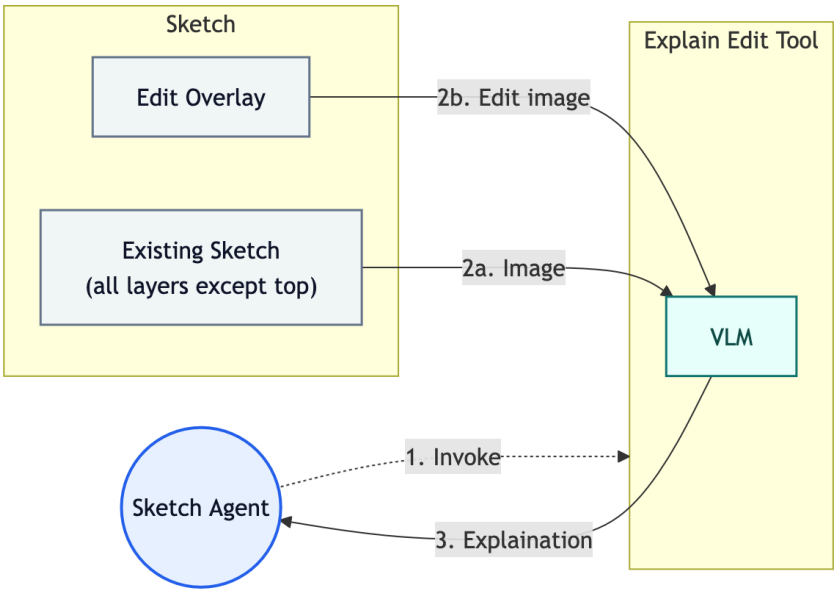


Figure 27. Explain Edit Tool

- `explain_edit` Directs a GPT-5 Nano VLM sub-agent to explain what the user intends to change. The Sketch Agent may provide an optional hint to aid the sub agent in its explanation, which is useful in situations where the user explicitly listed their intended edit.
    - The VLM Sub Agent is provided with images of the existing Sketch, composed of all layers except the top layer, and the user drawing overlay separately. This is necessary as the VLM may mistake the user's edit markings to be part of the actual drawing Sketch.
    - The Sub Agent is then tasked to explain the intention behind the user's drawing strokes. This text explanation is returned back to the Parent Sketch Agent to provide better understanding of the user's intended edit.

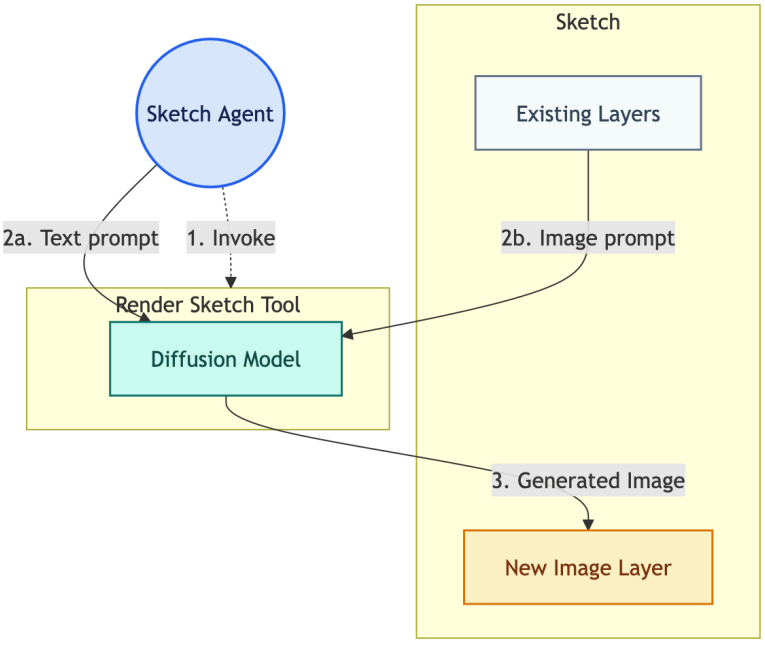


Figure 28. Render Sketch Tool

- **`render_sketch`** Generates a polished image from the Sketch by invoking the Flux.2 Klein 9B Diffusion Model. Based on the Sketch Agent's best understanding of the user's intended subject and style, it generates a text conditioning prompt. The text prompt, along with an image of the Sketch acting as image prompt, is fed to the Diffusion model to generate an image. This Image is appended to the user's Sketch as a new Image Layer.

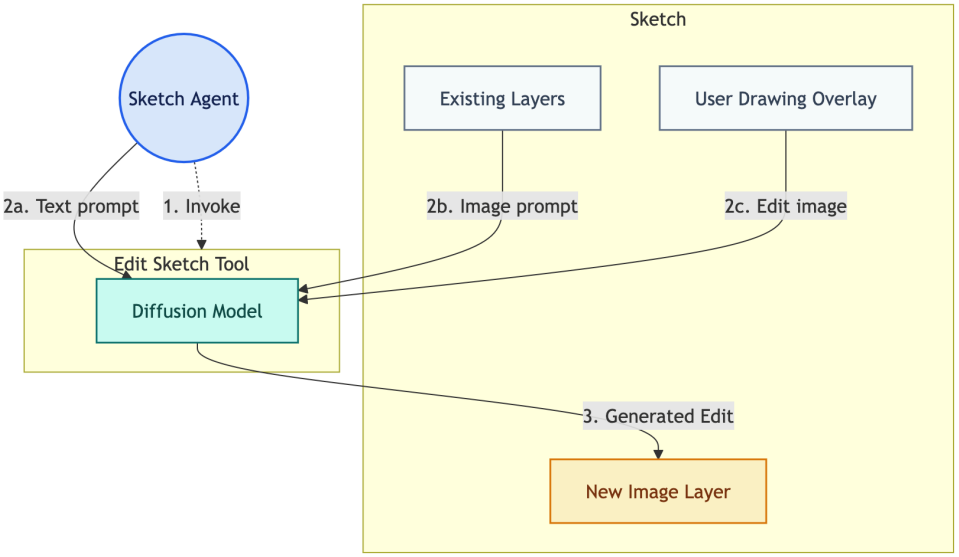


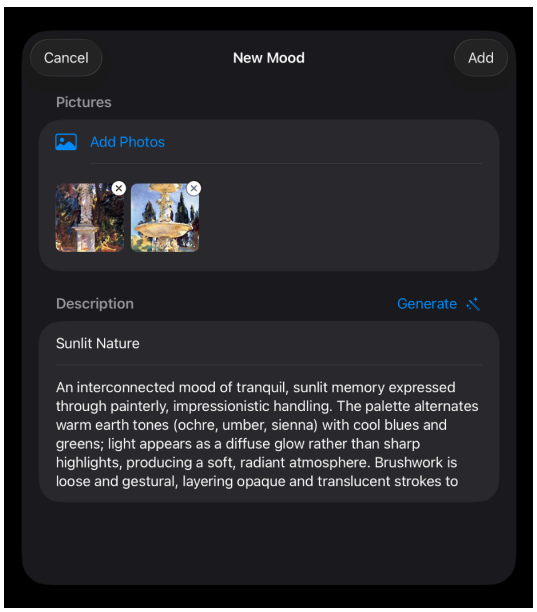
Figure 29. Edit Sketch Tool

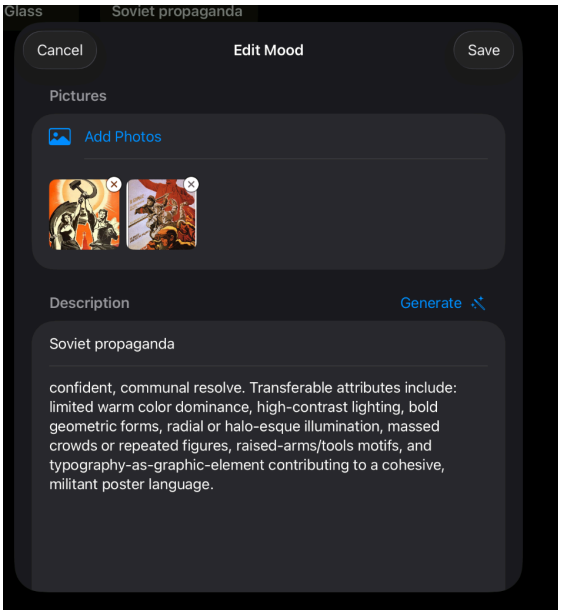
- **`edit_sketch`** Applies an incremental edit to the Sketch by invoking the Flux.2 Klein 9B Diffusion Model.
    - Based on the Sketch Agent's understanding of the user's intended edit, it generates a text conditioning prompt to perform the edit.
    - Similar to the **`explain_edit`** tool, the Diffusion model is fed the existing Sketch and the user drawing as separate images. This is to prevent the Diffusion model from reproducing the user's edit markings into the final Edit image.
    - Together with the text prompt, the Diffusion model generates an image which is appended as a new Image Layer onto the user's Sketch.
- **`list_mood`** Lists all user-created Moods, returning each mood's unique ID and title. This tool is invoked by the Sketch Agent to discover what Moods are available.
- **`inject_mood`** Retrieves the full title and description of a specific mood by its ID, typically used after list_mood to obtain the mood description. This mood description informs the Sketch Agents understanding of thematic characteristics it should reproduce in the images that it generates.

5. Results

The Broche App was evaluated through three author-conducted case studies covering Illustration, Graphic Design and UI Design. For each workload, Moods were added to inform the App on our desired visual style by adding reference Images and generating Mood descriptions. A basic drawing of the desired composition was then drawn on the Sketch View. The Sketch Agent in the AI tab was engaged to then render an initial AI generated image. Subsequent drawing and dialogue with the Sketch Agent was focused refining the generated image into a finished piece. Each workload was completed in approximately 30 mins end to end, from conception to completion.

- **Illustration** Evaluates its ability to assist in the end to end creation of an artistic illustration of a Picnic in Nature.
- **Graphic Design** Evaluates its ability to create a Graphic Poster with the words “Never Give Up” in the style of Soviet propaganda.
- **UI Design** Evaluates its ability to create a realistic looking UI design from a hastily drawn out UI wireframe.





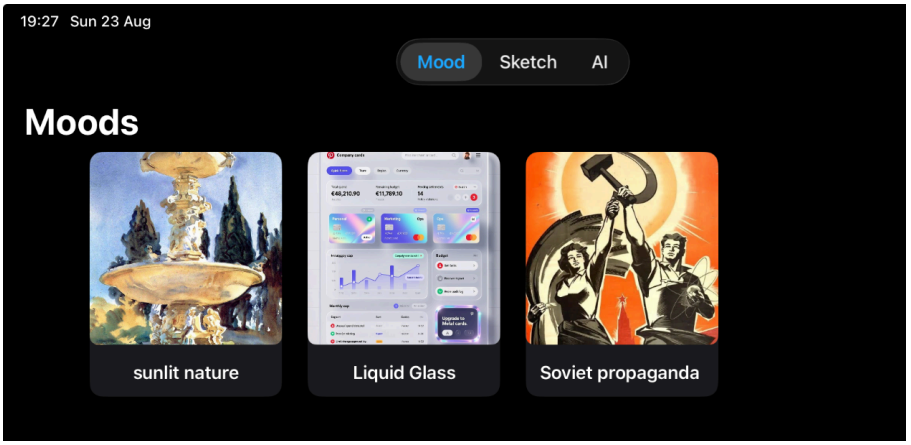
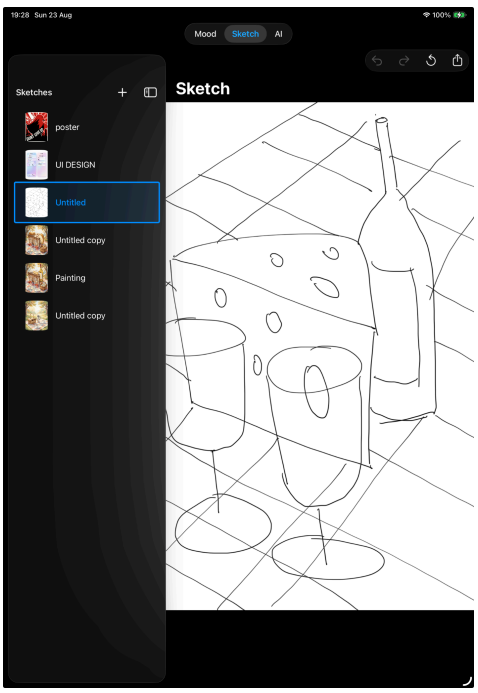
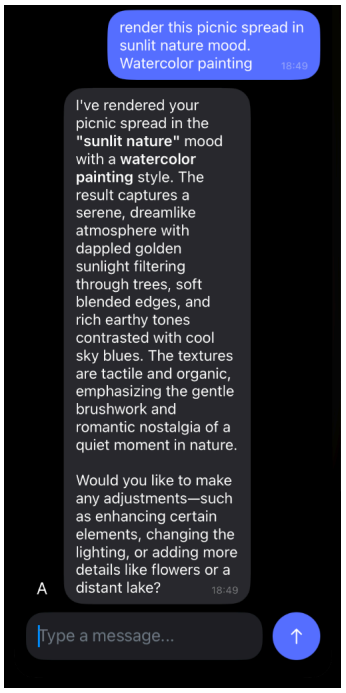
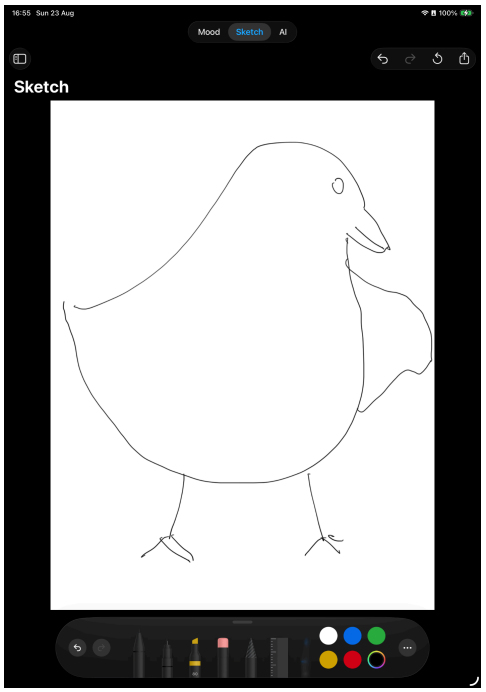


Figure 30. Adding a “Sunlit Nature” Mood with VLM generated description
Figure 31. Editing a “Soviet propaganda” Mood with VLM generated description
Figure 32. Mood Board with Moods added for Evaluation







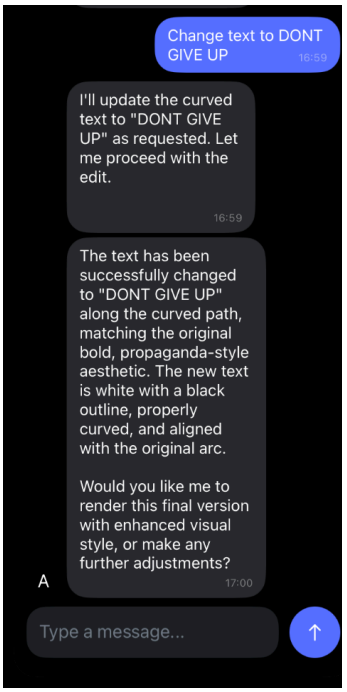


Figure 33. Sketch Drawn for Illustration Workload
Figure 34. Conversation with Sketch Agent for Illustration Workload to Render
Figure 35. Final Exported Graphic for Illustration Workload
Figure 36. Sketch Drawn for Graphic Design Workload
Figure 37. Conversation with Sketch Agent for Graphic Design Workload to Edit
Figure 38. Final Exported Graphic for Graphic Design Workload

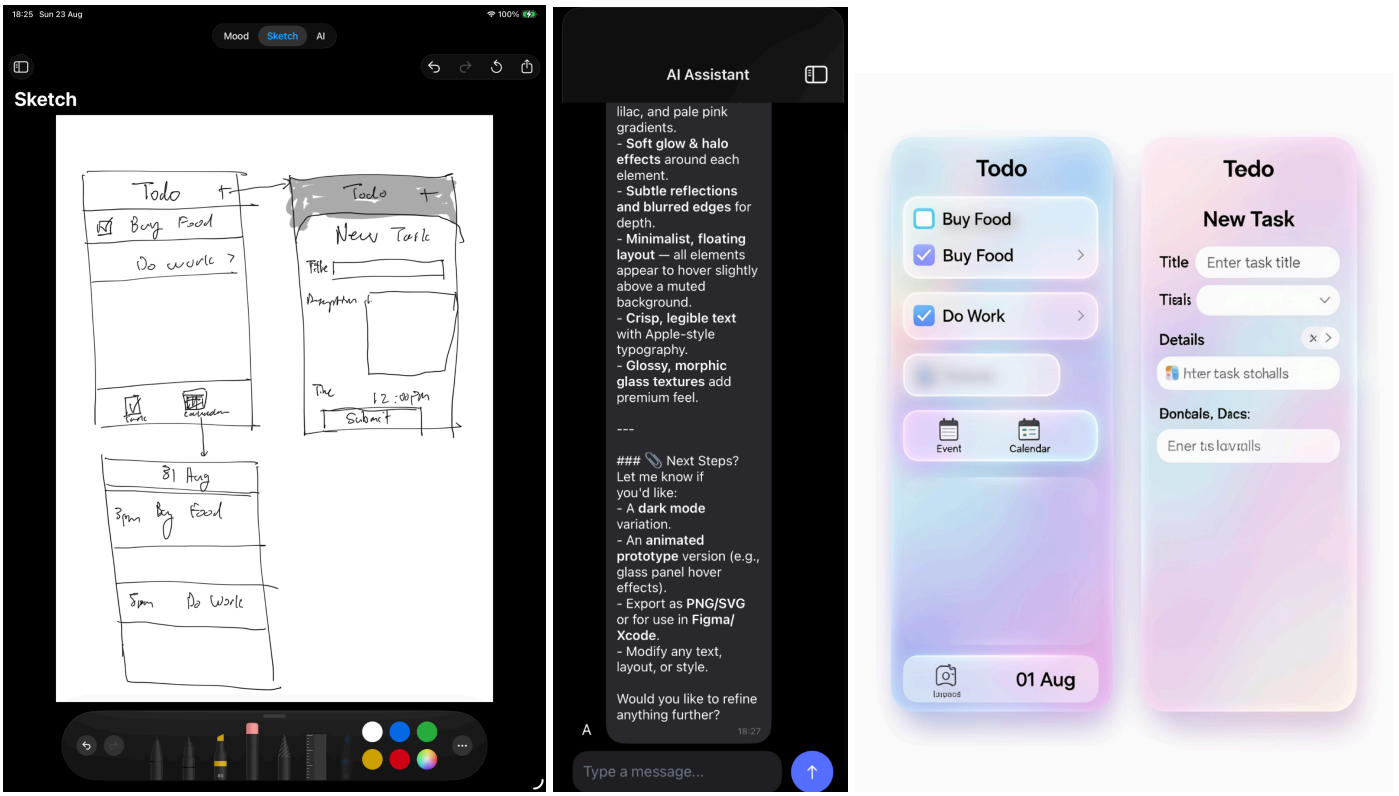


Figure 39. Sketch Drawn for UI Design Workload

Figure 40. Conversation with Sketch Agent for UI Design Workload to Edit

Figure 41. Final Exported Graphic for UI Design Workload

6. Discussion

6.1. Implications

Broche's introduction of a workflow for AI-Artist co-creation to complement the traditional artistic workflow for various workloads has two implications:

- **Accelerating Creativity** The AI Assisted workflow offered by the Broche App is likely faster than most artistic workflow with traditional methods. Although the time taken to complete a piece varies from artist to artist, the AI Assisted Workflow is likely faster since the artist is able to offload the meticulous rendering process to the Image Generation Model.
- **Aligned AI Assistance** AI generated content was generally more aligned with User intentions with the use of Caption Tool and injected Mood descriptions with some exceptions. These methods could be used to achieve better AI Artist Intention alignment.

6.2. Limitations

While Broche generally helps the artistic process there are several limitations that limit its use in real world artist work:

- **Lack of Control** The App sacrifices artistic control over the final piece each time AI generated content is incorporated into the piece. Artists are no longer able to choose the exact colours, textures, edges in their final work.
- **Hallucination** The AI Sketch Agent makes assumptions about user intentions in user messages and drawings. When these assumptions are misaligned with actual user expectations, the AI will generate something different from what the user originally intended.
    - In the Illustration Final Graphic, the box in the center of the picnic mat was originally intended to be rendered as a large piece of cheese.
    - In the UI Design Final Graphic, the "Todo" and "31 Aug" screen frames are intended to separate, but the image model has morphed them into a single screen frame.
- **Text Rendering** The Flux.2 Klein 9B model tends to clobber small text when generating images. This can be observed in the UI Design Final Graphic in clobbered rendering of "Enter Task Details".
- **Edge Truncation** Subjects generated by the Flux.2 Klein model tended to be truncated as they encroach on the edges of the image. This forces artists to place subjects within the frame of the image, even if they intend for objects to expand out of frame.

Figure 42. "Edge Truncation" of Banana Stem Close to Edge

- **Limited Drawing Tools** Broche relies on the default set of drawing tools available in PencilKit. Traditional drawing apps like Procreate offer significantly more variety in brushes.

- **No Layer Management UI** Users cannot manually arrange, remove or otherwise manage the layers of their Sketch as they may be able to do in drawing apps like Procreate.
- **Text Mood Descriptions** Moods are currently represented to the Sketch Agent primarily through generated textual descriptions. This limits the extent to which visual characteristics such as composition, texture and colour relationships can be conveyed.

A limitation of the current evaluation is that it only demonstrates three author-conducted workloads without a controlled baseline or user study, making it difficult to draw quantitative conclusions about Broche's effectiveness or generalise its benefits to other artists.

6.3. Future Work

Future work on AI-Assisted creation workflows can focus on improving on these limitations. New workflows to conduct fine grain editing with AI models could be designed to give artists the control they need. Prompt adherence, user intention awareness AI hallucination issues may be enhanced with better clarification workflows where the Agent asks more clarifying questions, to clarify its understanding of user intentions.

The Text Rendering and Edge Truncation issues can be rectified with by training improved Image Generation Models to work around these weaknesses. Research into enhanced Text Rendering capabilities is already underway with the latest Qwen Image models making strides into clear Text Rendering [16]. As the saying goes, an image is worth a thousand words. Moods may be better conveyed to Image Generation models though images rather than just text, supported by the recent addition of multi-image reference capabilities to these models [4].

A more systematic evaluation of Broche could be conducted by comparing its AI-assisted workflow against a traditional artistic workflow. Metrics such as task completion time, number of iterations, output quality and perceived artistic control could be measured across the Illustration, Graphic Design and UI Design workloads. This would provide stronger evidence on whether AI-assisted drawing applications like Broche actually accelerates artistic creation while maintaining user creative control.

Finally, the drawing capabilities of the App could be improved with the addition of more drawing tools. A layer management UI could also be added to allow users to reorder, hide or otherwise hide layers. This serves to further narrow the drawing functionality gap that artists expect from traditional drawing apps like Procreate.

7. Conclusion

Recent advances in AI image generation have put the role of Artists in question. The ability of modern AI models to generate high-fidelity images from text and visual prompts has raised questions about whether human artists remain necessary in the creative process. This project explored an alternative to replacing the artist: AI-Artist co-creation, where AI is used to augment rather than replace human creativity. Through the design and development of the Broche iPad AI-Assisted Drawing App , this project demonstrates that such a workflow is both technically feasible and practically useful for accelerating artistic creation.

Overall, although recent work on on-device AI inference runtimes demonstrates growing research interest in offline deployment, current models remain impractical to run on resource-constrained devices such as the iPad without alteration. Limited memory, compute resources, model support and long model loading times constrain the quality and usability of offline deployment. Online deployment was therefore selected for the App as it provides access to larger and more capable models.

The App's AI-assisted creative workflow is demonstrated across three graphical workloads: Illustration, Graphic Design and UI Design. In each workload, the artist was able to iteratively co-create with Sketch Agent that has access to AI Assisted Editing and Rendering tools powered by AI Image Generation models. Each workload was completed in approximately 30 minutes, demonstrating the potential of AI-assisted workflows to substantially reduce the time required to progress from an initial concept to a finished visual piece.

Despite these results, Broche has several limitations. AI-generated content reduces the artist's control over fine-grained visual details, while hallucination, text rendering and edge truncation can result in outputs that differ from the user's intentions. The application also has fewer drawing tools and layer management capabilities than established drawing applications. Future work could address these limitations.

Overall, the project demonstrates that generative AI can be integrated into existing artistic workflows without removing the artist from the creative process. By combining human direction with AI-assisted rendering and editing, the project demonstrates a practical model of augmenting rather than replacing artists with generative AI.

8. References

[1] J. van Hees, T. Grootswagers, G. L. Quek, and M. Varlet, “Human perception of art in the age of artificial intelligence,” *Front. Psychol.*, vol. 15, Jan. 2025, doi: 10.3389/fpsyg.2024.1497469.

[2] K. Roose, “AI-Generated Art Won a Prize. Artists Aren’t Happy,” *N. Y. Times*, Sep. 2022.

[3] Christies, “Obvious and the interface between art and artificial intelligence.” Accessed: Feb. 05, 2026. [Online]. Available: https://www.christies.com/en/stories/a-collaboration-between-two-artists-one-human-one-a-machine-0cd01f4e232f4279a525a446d60d4cd1

[4] B. F. Labs, “FLUX.2 [klein]: Towards Interactive Visual Intelligence.” Accessed: Aug. 19, 2026. [Online]. Available: https://bfl.ai/blog/flux2-klein-towards-interactive-visual-intelligence

[5] United States Copyright Office Review Board and T. S. Pester, “Second Request for Reconsideration for Refusal to Register Théâtre D’opéra Spatial,” U.S. Copyright Office, Washington, DC, Sep. 2023. [Online]. Available: https://www.copyright.gov/rulings-filings/review-board/docs/Theatre-Dopera-Spatial.pdf

[6] A. Hern and D. Milmo, “Spam, junk … slop? The latest wave of AI behind the ‘zombie internet,’” *The Guardian*, May 19, 2024. Accessed: Jan. 17, 2026. [Online]. Available: https://www.theguardian.com/technology/article/2024/may/19/spam-junk-slop-the-latest-wave-of-ai-behind-the-zombie-internet

[7] C. Anscomb, “AI: artistic collaborator?,” *AI Soc.*, vol. 40, no. 5, pp. 3419–3429, Jun. 2025, doi: 10.1007/s00146-024-02083-y.

[8] “ComfyUI Official Documentation,” ComfyUI. Accessed: Aug. 19, 2026. [Online]. Available: https://docs.comfy.org

[9] “Draw Things: AI-assisted Image Generation.” Accessed: Aug. 19, 2026. [Online]. Available: https://drawthings.ai/

[10] “Draw Anything - Stable Diffusion Playground.” Accessed: Aug. 19, 2026. [Online]. Available: https://www.drawanything.app/

[11] “SageBrush: AI Painter, The Simplest Way to Create with AI.” Accessed: Aug. 19, 2026. [Online]. Available: https://www.apposite.ai/sagebrush-ai-painter-the-simplest-way-to-create-with-ai.html

[12] “Wand – Draw anything.” Accessed: Aug. 19, 2026. [Online]. Available: https://wand.app/

[13] “GPT Image Generation Models Prompting Guide,” OpenAI Developers. Accessed: Sep. 06, 2026. [Online]. Available: https://developers.openai.com/cookbook/examples/multimodal/image-gen-models-prompting-guide

[14] J. Betker *et al.*, “Improving Image Generation with Better Captions”.

[15] “My Process - How I Create an Illustration from Start to Finish.” Accessed: Aug. 19, 2026. [Online]. Available: https://www.linkedin.com/pulse/my-process-how-i-create-illustration-from-start-finish-bartlett

[16] C. Wu *et al.*, “Qwen-Image Technical Report,” Aug. 04, 2025, *arXiv*: arXiv:2508.02324. doi: 10.48550/arXiv.2508.02324.

[17] “Gestures — Procreate Handbook,” Procreate Help Center. Accessed: Aug. 23, 2026. [Online]. Available: https://help.procreate.com/procreate/handbook/interface-gestures/gestures

[18] R. Nakano *et al.*, “WebGPT: Browser-assisted question-answering with human feedback,” Jun. 01, 2022, *arXiv*: arXiv:2112.09332. doi: 10.48550/arXiv.2112.09332.

[19] J. Yang *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” Nov. 11, 2024, *arXiv*: arXiv:2405.15793. doi: 10.48550/arXiv.2405.15793.

[20] A. Vaswani *et al.*, “Attention Is All You Need,” Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.

[21] P. J. Liu *et al.*, “Generating Wikipedia by Summarizing Long Sequences,” Jan. 30, 2018, *arXiv*: arXiv:1801.10198. doi: 10.48550/arXiv.1801.10198.

[22] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models,” Jan. 23, 2020, *arXiv*: arXiv:2001.08361. doi: 10.48550/arXiv.2001.08361.

[23] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” Mar. 04, 2022, *arXiv*: arXiv:2203.02155. doi: 10.48550/arXiv.2203.02155.

[24] A. Radford *et al.*, “Learning Transferable Visual Models From Natural Language Supervision,” Feb. 26, 2021, *arXiv*: arXiv:2103.00020. doi: 10.48550/arXiv.2103.00020.

[25] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” Nov. 18, 2015, *arXiv*: arXiv:1503.03585. doi: 10.48550/arXiv.1503.03585.

[26] X. Liu, C. Gong, and Q. Liu, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow,” Sep. 07, 2022, *arXiv*: arXiv:2209.03003. doi: 10.48550/arXiv.2209.03003.

[27] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” Apr. 13, 2022, *arXiv*: arXiv:2112.10752. doi: 10.48550/arXiv.2112.10752.

[28] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” Jul. 26, 2022, *arXiv*: arXiv:2207.12598. doi: 10.48550/arXiv.2207.12598.

[29] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” Mar. 02, 2023, *arXiv*: arXiv:2212.09748. doi: 10.48550/arXiv.2212.09748.

[30] “LLM Leaderboard - Comparison of AI models from OpenAI, Anthropic, Google, SpaceXAI & others | Artificial Analysis.” Accessed: Aug. 24, 2026. [Online]. Available: https://artificialanalysis.ai/leaderboards/models
[31] A. Yang *et al.*, “Qwen3 Technical Report,” May 14, 2025, *arXiv*: arXiv:2505.09388. doi: 10.48550/arXiv.2505.09388.
[32] J. Lang, Z. Guo, and S. Huang, “A Comprehensive Study on Quantization Techniques for Large Language Models,” Oct. 30, 2024, *arXiv*: arXiv:2411.02530. doi: 10.48550/arXiv.2411.02530.
[33] “About | Artificial Analysis.” Accessed: Aug. 21, 2026. [Online]. Available: https://artificialanalysis.ai/about
[34] “Intelligence Benchmarking | Artificial Analysis.” Accessed: Aug. 21, 2026. [Online]. Available: https://artificialanalysis.ai/methodology/intelligence-benchmarking
[35] “Image Generation Benchmarking Methodology | Artificial Analysis.” Accessed: Aug. 21, 2026. [Online]. Available: https://artificialanalysis.ai/image/methodology
[36] H. Zhang, “SC3000 Lecture 1 Module 2_Intelligent Agents and Search,” 2025.
[37] X. Qu *et al.*, “A Comprehensive Review of AI Agents: Transforming Possibilities in Technology and Beyond,” Aug. 16, 2025, *arXiv*: arXiv:2508.11957. doi: 10.48550/arXiv.2508.11957.
[38] “Claude Code by Anthropic | AI Coding Agent, Terminal, IDE,” Claude. Accessed: Aug. 29, 2026. [Online]. Available: https://claude.com/product/claude-code
[39] “Codex in ChatGPT | AI Coding Agents for Software Engineering,” OpenAI. Accessed: Aug. 29, 2026. [Online]. Available: https://openai.com/codex/
[40] “Google Antigravity,” Google Antigravity. Accessed: Aug. 29, 2026. [Online]. Available: https://antigravity.google/
[41] “Context windows,” Claude Platform Docs. Accessed: Aug. 25, 2026. [Online]. Available: https://platform.claude.com/docs/en/build-with-claude/context-windows
[42] *Full Walkthrough: Workflow for AI Coding — Matt Pocock*, (Apr. 24, 2026). Accessed: Aug. 25, 2026. [Online Video]. Available: https://www.youtube.com/watch?v=-QFHIoCo-Kof
[43] “Subagents,” ChatGPT Learn. Accessed: Aug. 25, 2026. [Online]. Available: https://learn.chatgpt.com/docs/agent-configuration/subagents
[44] I. Sommerville, *Software engineering*, Tenth edition. in Always learning. Boston Columbus Indianapolis New York San Francisco Hoboken Amsterdam Cape Town Dubai London: Pearson, 2016.
[45] E. Gamma, Ed., *Design patterns: elements of reusable object-oriented software*, 39. printing. in Addison-Wesley professional computing series. Boston, Mass. Munich: Addison-Wesley, 2011.
[46] F. Lardinois, “Apple Launches Swift, A New Programming Language For Writing iOS And OS X Apps,” TechCrunch. Accessed: Aug. 20, 2026. [Online]. Available: https://techcrunch.com/2014/06/02/apple-launches-swift-a-new-programming-language-for-writing-ios-and-os-x-apps/
[47] A. Inc, “SwiftUI,” Apple Developer. Accessed: Aug. 20, 2026. [Online]. Available: https://developer.apple.com/swiftui/
[48] “SwiftData,” Apple Developer Documentation. Accessed: Aug. 20, 2026. [Online]. Available: https://developer.apple.com/documentation/swiftdata
[49] “CloudKit,” Apple Developer Documentation. Accessed: Aug. 20, 2026. [Online]. Available: https://developer.apple.com/documentation/cloudkit/
[50] “PencilKit,” Apple Developer Documentation. Accessed: Aug. 20, 2026. [Online]. Available: https://developer.apple.com/documentation/pencilkit
[51] “GitHub - exyte/Chat: A SwiftUI Chat UI framework with fully customizable message cells and a built-in media picker · GitHub.” Accessed: Aug. 20, 2026. [Online]. Available: https://github.com/exyte/Chat
[52] *MacPaw/OpenAI*. (Aug. 19, 2026). Swift. MacPaw Inc. Accessed: Aug. 20, 2026. [Online]. Available: https://github.com/MacPaw/OpenAI
[53] *replicate/replicate-swift*. (Jul. 31, 2026). Swift. Replicate. Accessed: Aug. 20, 2026. [Online]. Available: https://github.com/replicate/replicate-swift
[54] “Firebase | Google’s Mobile and Web App Development Platform.” Accessed: Aug. 28, 2026. [Online]. Available: https://firebase.google.com/
[55] “Global Infrastructure,” Google Cloud. Accessed: Aug. 28, 2026. [Online]. Available: https://cloud.google.com/infrastructure
[56] “How does Replicate work? - Replicate.” Accessed: Aug. 20, 2026. [Online]. Available: https://replicate.com/docs/reference/how-does-replicate-work
[57] “OpenRouter Quickstart Guide,” OpenRouter Documentation. Accessed: Aug. 20, 2026. [Online]. Available: https://openrouter.ai/docs/quickstart
[58] “The Rising Risk Of Exposed ChatGPT API Keys.” Accessed: Aug. 28, 2026. [Online]. Available: https://cyble.com/blog/when-ai-secrets-go-public-chatgpt/
[59] “Google API Keys Weren’t Secrets. But then Gemini Changed the Rules. ◆ Truffle Security Co.” Accessed: Aug. 28, 2026. [Online]. Available: https://trufflesecurity.com/blog/google-api-keys-werent-secrets-but-then-gemini-changed-the-rules
[60] “BYOK Meaning: What ‘Bring Your Own Key’ Is and Why It’s the Future of AI Tools.” Accessed: Aug. 28, 2026. [Online]. Available: https://surfmind.ai/blog/byok-bring-your-own-key-future-of-ai-tools

[61] “Foundation Models,” Apple Developer Documentation. Accessed: Aug. 21, 2026. [Online]. Available: https://developer.apple.com/documentation/foundationmodels
[62] “MLX.” Accessed: Aug. 21, 2026. [Online]. Available: https://mlx-framework.org/#features
[63] *ml-explore/mlx-swift-lm*. (Aug. 19, 2026). Swift. ml-explore. Accessed: Aug. 20, 2026. [Online]. Available: https://github.com/ml-explore/mlx-swift-lm
[64] “Google AI Edge,” Google for Developers. Accessed: Aug. 21, 2026. [Online]. Available: https://developers.google.com/edge/litert-lm
[65] “GitHub - trymirai/uzu: A high-performance inference engine for AI models · GitHub.” Accessed: Aug. 21, 2026. [Online]. Available: https://github.com/trymirai/uzu
[66] A. Orhon, M. Siracusa, and A. Wadhwa, *Stable Diffusion with Core ML on Apple Silicon*. (2022). [Online]. Available: https://github.com/apple/ml-stable-diffusion
[67] *drawthingsai/media-generation-kit*. (Aug. 25, 2026). Swift. Draw Things. Accessed: Aug. 26, 2026. [Online]. Available: https://github.com/drawthingsai/media-generation-kit
[68] Z. Zhanyan, “DA5005 Final Redux Report.”
[69] “About Pinterest | Find, shop and live what you love,” Pinterest. Accessed: Aug. 25, 2026. [Online]. Available: https://about.pinterest.com/
[70] “My Creative Workflow from Start to Finish - MEGHANN RADER ILLUSTRATION.” Accessed: Aug. 19, 2026. [Online]. Available: https://www.meghannrader.com/blog/my-creative-workflow-from-start-to-finish
[71] F. Webb, *The artist’s guide to composition: how to design eye-catching paintings in all mediums*. Newton Abbot: David & Charles, 1995.
[72] “Maximum build file sizes - App uploads - Reference - App Store Connect - Help - Apple Developer.” Accessed: Aug. 26, 2026. [Online]. Available: https://developer.apple.com/help/app-store-connect/reference/app-uploads/maximum-build-file-sizes
[73] “Firebase Pricing,” Firebase. Accessed: Aug. 28, 2026. [Online]. Available: https://firebase.google.com/pricing
[74] Z. Zhanyan, *mrzzy/exyte-Chat*. (Jul. 20, 2026). Swift. Accessed: Aug. 22, 2026. [Online]. Available: https://github.com/mrzzy/exyte-Chat
[75] “Qwen3 30B A3B Instruct 2507 - API Pricing & Benchmarks | OpenRouter.” Accessed: Aug. 28, 2026. [Online]. Available: https://openrouter.ai/qwen/qwen3-30b-a3b-instruct-2507
[76] “GPT-5 Nano - API Pricing & Benchmarks.” Accessed: Aug. 28, 2026. [Online]. Available: https://openrouter.ai/openai/gpt-5-nano
[77] “FLUX.2 klein 9B | Image Generation API.” Accessed: Aug. 28, 2026. [Online]. Available: https://replicate.com/black-forest-labs/flux-2-klein-9b
[78] “OpenRouter API Reference - Complete Documentation,” OpenRouter Documentation. Accessed: Aug. 29, 2026. [Online]. Available: https://openrouter.ai/docs/api_reference/overview